

Κώδικας Πειραματικού Σταδίου Ανάπτυξης CNN

```
In [8]: from IPython.display import Image
import os
#suppressing tensorflow warnings
os.environ["GRPC_VERBOSITY"] = "ERROR"
os.environ["GLOG_minloglevel"] = "2"
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
os.environ["CUDA_VISIBLE_DEVICES"] = "0"

# importing tensorflow and keras
import tensorflow as tf

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

import pathlib

tf.get_logger().setLevel("ERROR")
```

Loading the Datasets

```
In [9]: # Load Dataset
data_dir = pathlib.Path("images/Cyrillic_train")
image_count = len(list(data_dir.glob("*/*.png")))
batch_size = 32
img_height = 32
img_width = 32
num_classes = 34
# size of pooling area for max pooling
pool_size = (2, 2)
epochs = 15
epochs_range = range(epochs) # needed for plotting the training and validation

print("Loading training and validation datasets...")
training_dataset = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    subset="training",
    validation_split=0.2,
    color_mode="rgba", # VERY IMPORTANT AS WE ARE DEALING WITH IMAGES WITH
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    labels="inferred",
    label_mode="categorical",
```

```

        shuffle=True, # already default but just to be explicit, we want to shuffle
        verbose=1,
    )

validation_dataset = tf.keras.utils.image_dataset_from_directory(
    data_dir,
    subset="validation",
    validation_split=0.2,
    color_mode="rgb", # VERY IMPORTANT AS WE ARE DEALING WITH IMAGES WITH
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    labels="inferred",
    label_mode="categorical",
    shuffle=False, # we don't want to shuffle the validation dataset
    verbose=1,
)

# Load the test dataset
print("Loading test dataset...")
test_data_dir = pathlib.Path("images/Cyrillic_test")
test_dataset = tf.keras.utils.image_dataset_from_directory(
    test_data_dir,
    color_mode="rgb",
    seed=123,
    image_size=(img_height, img_width),
    batch_size=batch_size,
    labels="inferred",
    label_mode="categorical",
    # we DON'T want to shuffle the test dataset because we want two passes to
    shuffle=False,
    verbose=1,
)

```

```

Loading training and validation datasets...
Found 10819 files belonging to 34 classes.
Using 8656 files for training.
Found 10819 files belonging to 34 classes.
Using 2163 files for validation.
Loading test dataset...
Found 4661 files belonging to 34 classes.

```

```

In [10]: class_names = training_dataset.class_names
         print(class_names)

```

```

['I', 'E', 'A', 'Б', 'В', 'Г', 'Д', 'Е', 'Ж', 'З', 'И', 'Й', 'К', 'Л', 'М',
'H', 'O', 'П', 'Р', 'С', 'Т', 'У', 'Ф', 'Х', 'Ц', 'Ч', 'Ш', 'Щ', 'Ъ', 'Ы',
'ь', 'Э', 'Ю', 'Я']

```

```

In [11]: for image_batch, labels_batch in training_dataset:
         print(image_batch.shape)
         print(labels_batch.shape)
         break

```

```

(32, 32, 32, 4)
(32, 34)

```

```

In [12]: folder_counts = {}
for subdir in sorted(data_dir.iterdir()):
    if subdir.is_dir():
        count = sum(1 for file in subdir.iterdir() if file.is_file())
        folder_counts[subdir.name] = count

for folder, count in folder_counts.items():
    print(f"{folder}: {count}")

print(f"Total images in {data_dir}: {sum(folder_counts.values())}")

# Bar charts for folder_counts
labels = list(folder_counts.keys())
counts = list(folder_counts.values())

plt.figure(figsize=(14, 6))
sns.barplot(x=labels, y=counts, palette="viridis")
plt.xticks(rotation=90)
plt.title(f"Image counts per class in {data_dir}")
plt.xlabel("Class")
plt.ylabel("Image Count")
for i, v in enumerate(counts):
    plt.text(i, v + max(counts)*0.01, str(v), ha="center", va="bottom", font
plt.tight_layout()
plt.show()

# Sorted (descending) horizontal bar chart for easier comparison
sorted_items = sorted(folder_counts.items(), key=lambda x: x[1], reverse=True)
sorted_labels, sorted_counts = zip(*sorted_items)

plt.figure(figsize=(8, 10))
sns.barplot(x=list(sorted_counts), y=list(sorted_labels), palette="magma")
plt.title("Image counts per class (sorted descending)")
plt.xlabel("Image Count")
plt.ylabel("Class")
for i, v in enumerate(sorted_counts):
    plt.text(v + max(sorted_counts)*0.01, i, str(v), va="center", fontsize=8
plt.tight_layout()
plt.show()

```

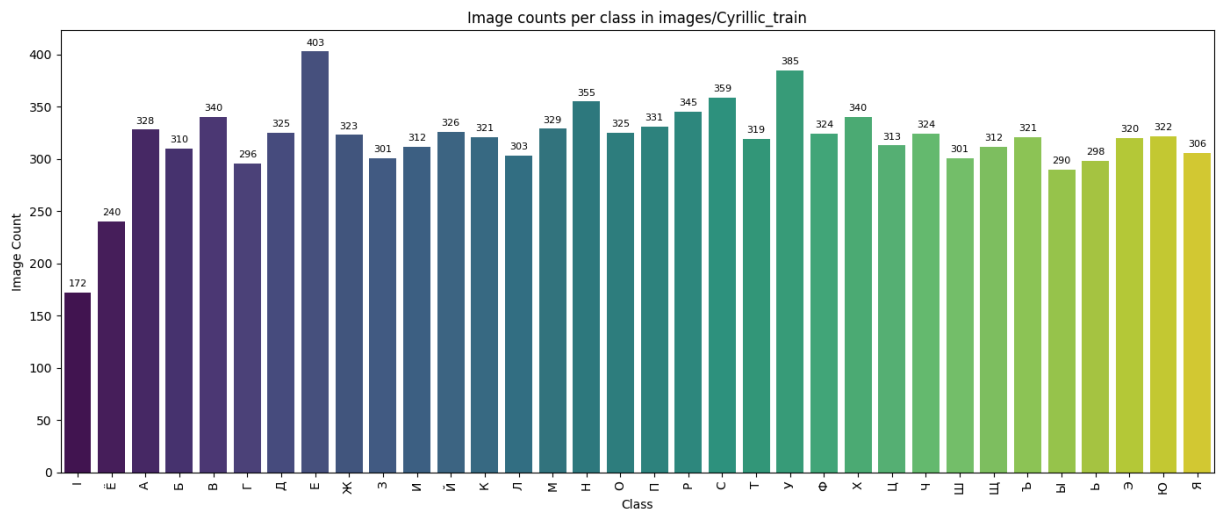
I: 172
Ě: 240
A: 328
Б: 310
В: 340
Г: 296
Д: 325
Е: 403
Ж: 323
З: 301
И: 312
Й: 326
К: 321
Л: 303
М: 329
Н: 355
О: 325
П: 331
Р: 345
С: 359
Т: 319
У: 385
Ф: 324
Х: 340
Ц: 313
Ч: 324
Ш: 301
Щ: 312
Ъ: 321
Ы: 290
Ь: 298
Э: 320
Ю: 322
Я: 306

Total images in images/Cyrillic_train: 10819

/tmp/ipykernel_6528/1174008785.py:17: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

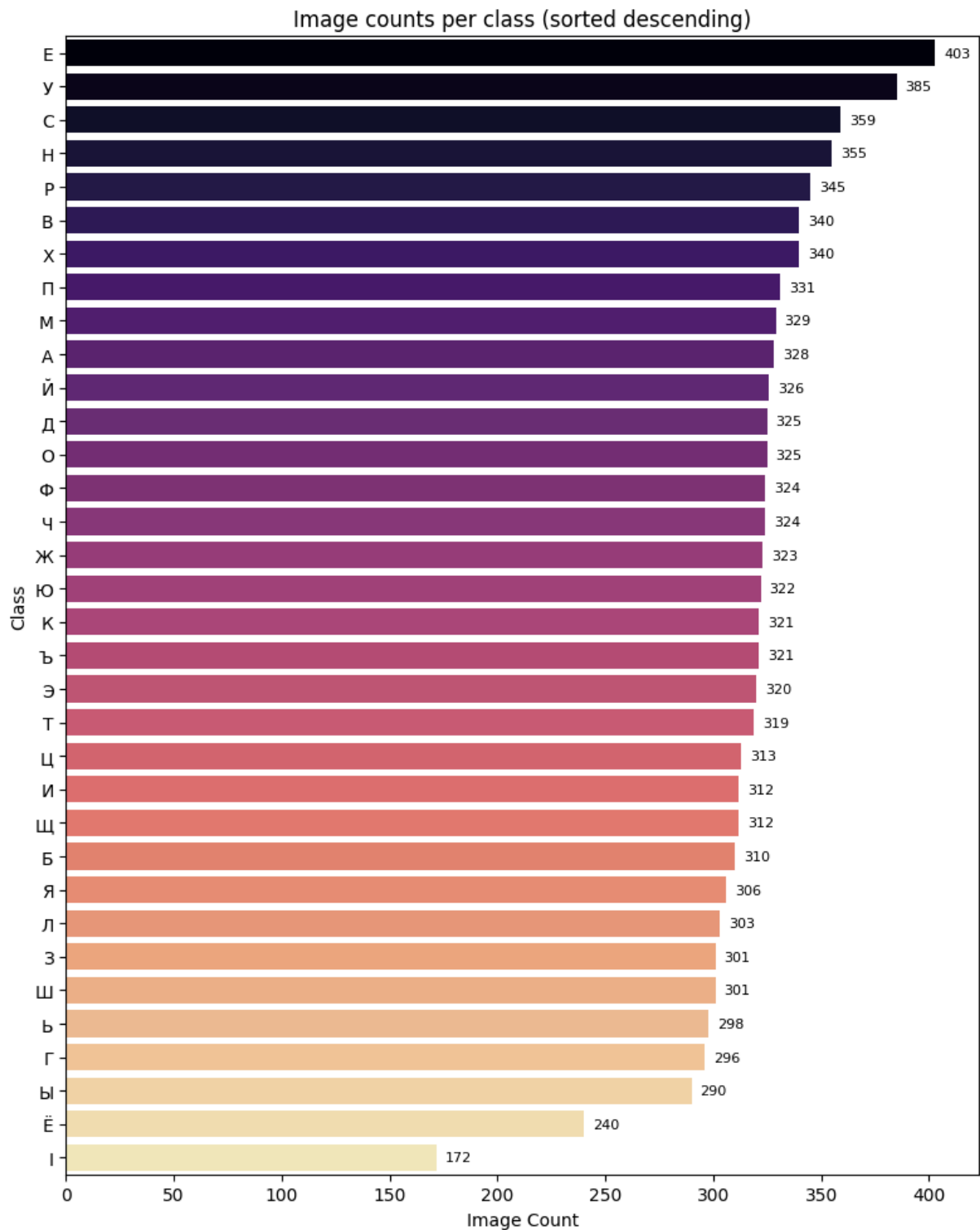
```
sns.barplot(x=labels, y=counts, palette="viridis")
```



/tmp/ipykernel_6528/1174008785.py:32: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=list(sorted_counts), y=list(sorted_labels), palette="magma")
```



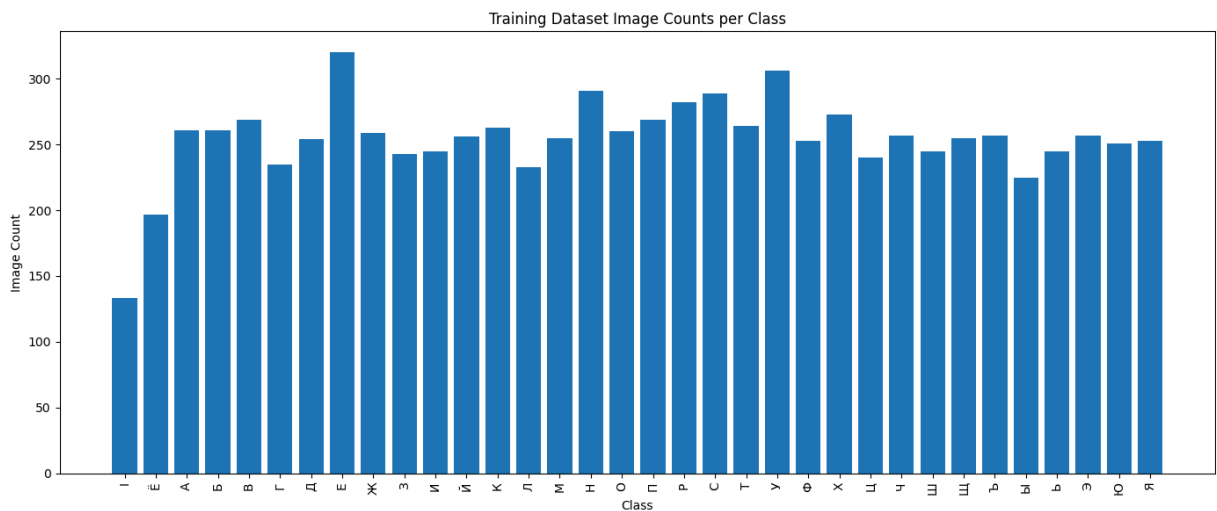
```
In [13]: train_labels = np.concatenate(
            [np.argmax(labels_batch, axis=1) for _, labels_batch in training_dataset
            ])
train_counts = np.bincount(train_labels, minlength=len(class_names))

for class_name, count in zip(class_names, train_counts):
    print(f"{class_name}: {count}")

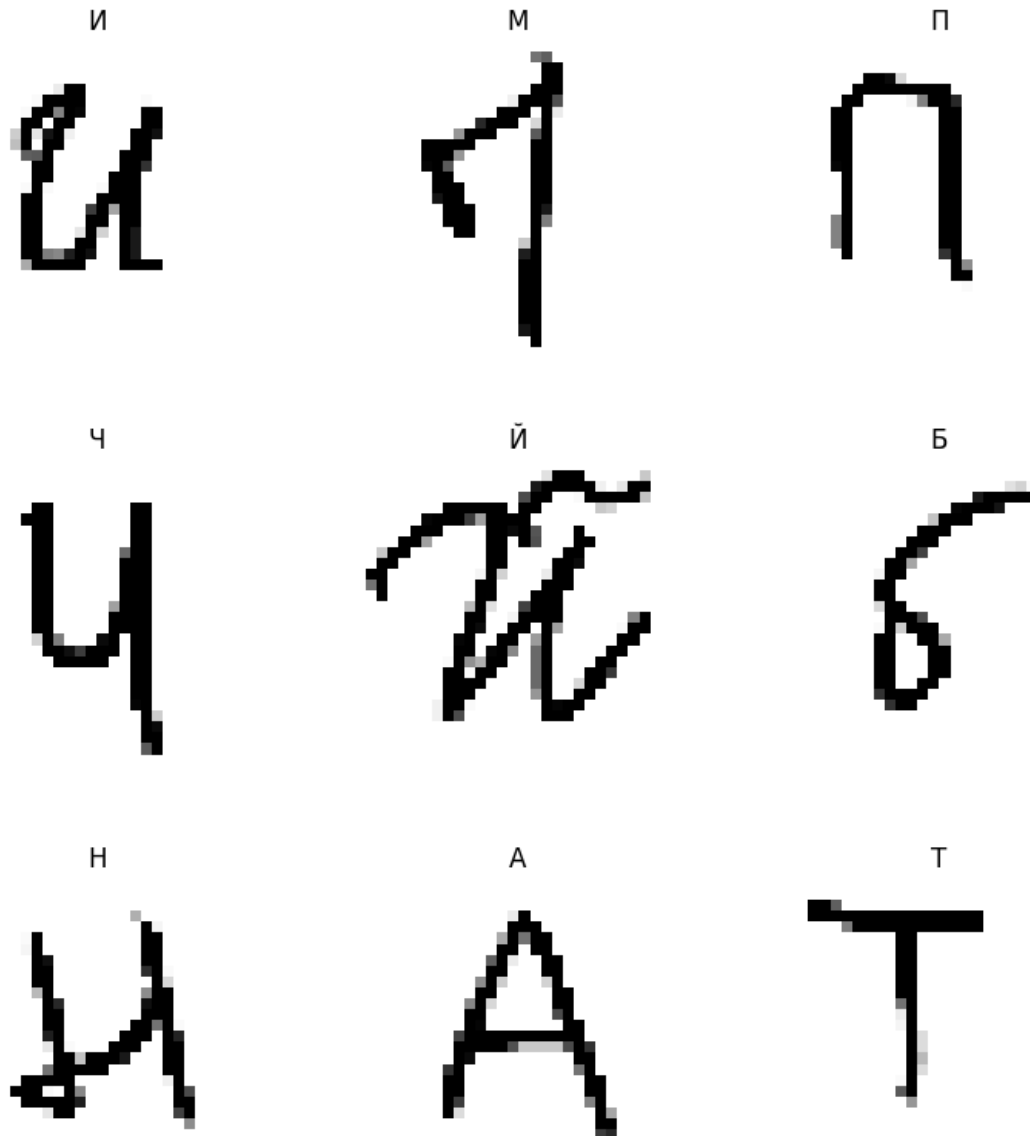
plt.figure(figsize=(14, 6))
plt.bar(class_names, train_counts)
```

```
plt.xticks(rotation=90)
plt.title("Training Dataset Image Counts per Class")
plt.ylabel("Image Count")
plt.xlabel("Class")
plt.tight_layout()
plt.show()
```

I: 133
 Ě: 197
 A: 261
 Б: 261
 В: 269
 Г: 235
 Д: 254
 Е: 320
 Ж: 259
 З: 243
 И: 245
 Й: 256
 К: 263
 Л: 233
 М: 255
 Н: 291
 О: 260
 П: 269
 Р: 282
 С: 289
 Т: 264
 У: 306
 Ф: 253
 Х: 273
 Ц: 240
 Ч: 257
 Ш: 245
 Щ: 255
 Ъ: 257
 Ы: 225
 Ь: 245
 Э: 257
 Ю: 251
 Я: 253



```
In [14]: plt.figure(figsize=(10, 10))
for images, labels in training_dataset.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        # labels are one-hot vectors when using label_mode='categorical'
        # convert to integer index before using as an index into class_names
        plt.title(class_names[int(np.argmax(labels[i]))])
        plt.axis("off")
```



Training our Model

Raw Logits on Output, no Dropout, default 32 Batch


```
In [8]: model_main = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1./255),
    tf.keras.layers.Conv2D(32, 3, activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, activation='relu'),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(num_classes)
])
```

```
In [9]: model_main.compile(
    optimizer='adam',
    loss=tf.keras.losses.CategoricalCrossentropy(from_logits=True),
    metrics=['accuracy'])
```

```
In [10]: results_main_model = model_main.fit(
    training_dataset,
    validation_data=validation_dataset,
    epochs=epochs,
    batch_size=32,
    verbose=1
)
```

Epoch 1/15

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1782474862.942086 22987 service.cc:146] XLA service 0x76b7a40054c0 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:

I0000 00:00:1782474862.942144 22987 service.cc:154] StreamExecutor device (0): NVIDIA GeForce GTX 1050 Ti, Compute Capability 6.1

16/271 ————— **2s** 11ms/step - accuracy: 0.0496 - loss: 3.5194

I0000 00:00:1782474865.905204 22987 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.

271/271  **16s** 42ms/step - accuracy: 0.4331 - loss: 2.0349
 - val_accuracy: 0.7688 - val_loss: 0.8194
 Epoch 2/15
271/271  **3s** 11ms/step - accuracy: 0.7497 - loss: 0.8551
 - val_accuracy: 0.8618 - val_loss: 0.4674
 Epoch 3/15
271/271  **3s** 10ms/step - accuracy: 0.8187 - loss: 0.6114
 - val_accuracy: 0.8706 - val_loss: 0.4541
 Epoch 4/15
271/271  **3s** 10ms/step - accuracy: 0.8535 - loss: 0.4822
 - val_accuracy: 0.8715 - val_loss: 0.3963
 Epoch 5/15
271/271  **3s** 11ms/step - accuracy: 0.8759 - loss: 0.4063
 - val_accuracy: 0.9246 - val_loss: 0.2590
 Epoch 6/15
271/271  **3s** 10ms/step - accuracy: 0.8929 - loss: 0.3495
 - val_accuracy: 0.9404 - val_loss: 0.2120
 Epoch 7/15
271/271  **2s** 9ms/step - accuracy: 0.9095 - loss: 0.2866 -
 val_accuracy: 0.9501 - val_loss: 0.1751
 Epoch 8/15
271/271  **2s** 9ms/step - accuracy: 0.9232 - loss: 0.2513 -
 val_accuracy: 0.9177 - val_loss: 0.2518
 Epoch 9/15
271/271  **2s** 9ms/step - accuracy: 0.9328 - loss: 0.2145 -
 val_accuracy: 0.9052 - val_loss: 0.2676
 Epoch 10/15
271/271  **2s** 9ms/step - accuracy: 0.9421 - loss: 0.1768 -
 val_accuracy: 0.9459 - val_loss: 0.1657
 Epoch 11/15
271/271  **2s** 9ms/step - accuracy: 0.9503 - loss: 0.1556 -
 val_accuracy: 0.9612 - val_loss: 0.1228
 Epoch 12/15
271/271  **3s** 10ms/step - accuracy: 0.9567 - loss: 0.1267
 - val_accuracy: 0.9501 - val_loss: 0.1567
 Epoch 13/15
271/271  **3s** 10ms/step - accuracy: 0.9605 - loss: 0.1115
 - val_accuracy: 0.9709 - val_loss: 0.1048
 Epoch 14/15
271/271  **3s** 9ms/step - accuracy: 0.9707 - loss: 0.0883 -
 val_accuracy: 0.9519 - val_loss: 0.1771
 Epoch 15/15
271/271  **2s** 9ms/step - accuracy: 0.9722 - loss: 0.0829 -
 val_accuracy: 0.9676 - val_loss: 0.1239

Training Results

Generate the confusion matrices

```

In [11]: # Compute predictions and true labels
y_pred_probs = model_main.predict(test_dataset)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.concatenate([np.argmax(labels_batch, axis=1) for _, labels_batch

```

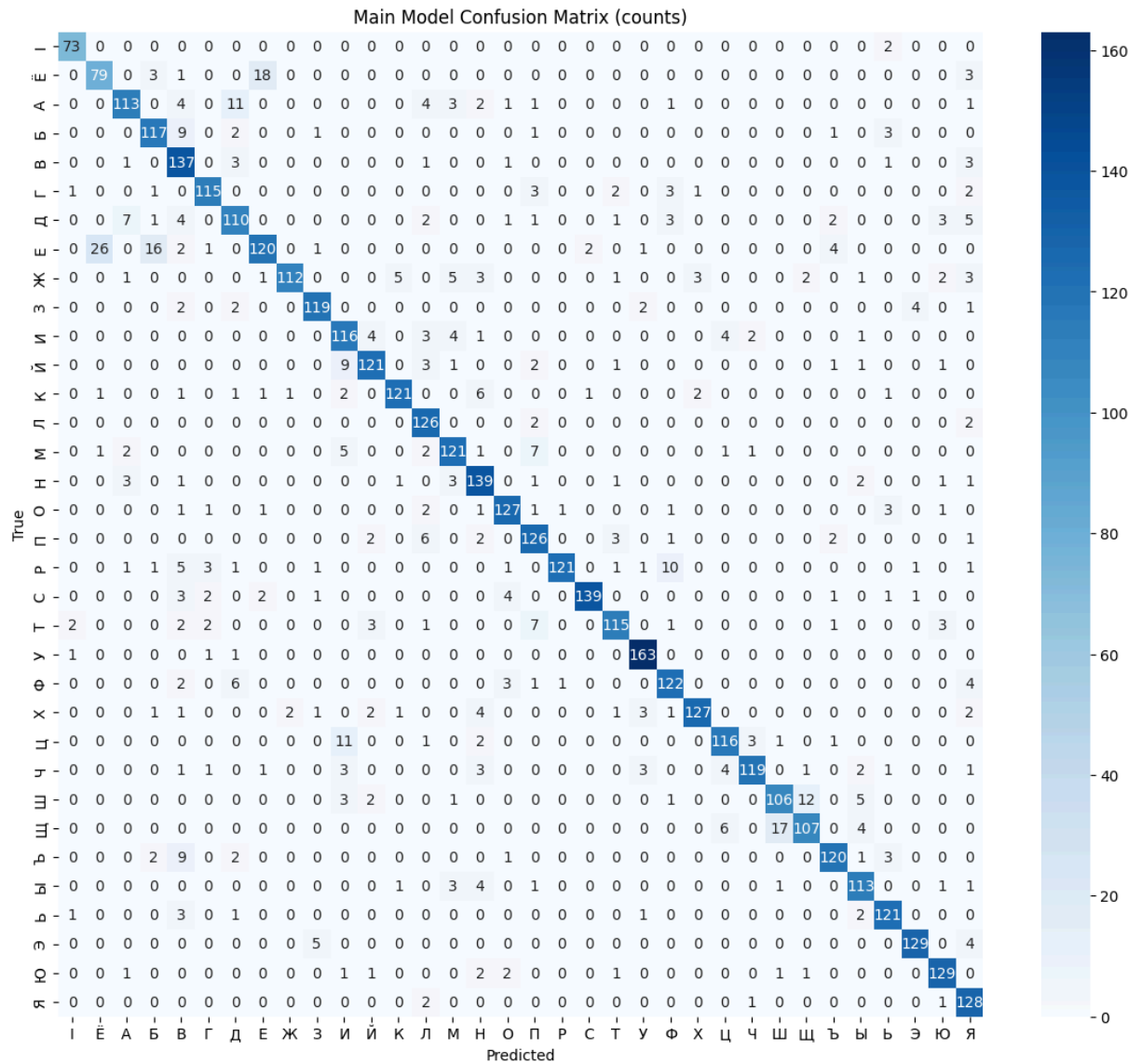
```

# Confusion matrix and classification report
cm = confusion_matrix(y_true, y_pred)
print(classification_report(y_true, y_pred, target_names=class_names, zero_c

# Plot confusion matrix (raw counts)
plt.figure(figsize=(14, 12))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_names,
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Main Model Confusion Matrix (counts)")
plt.show()

```

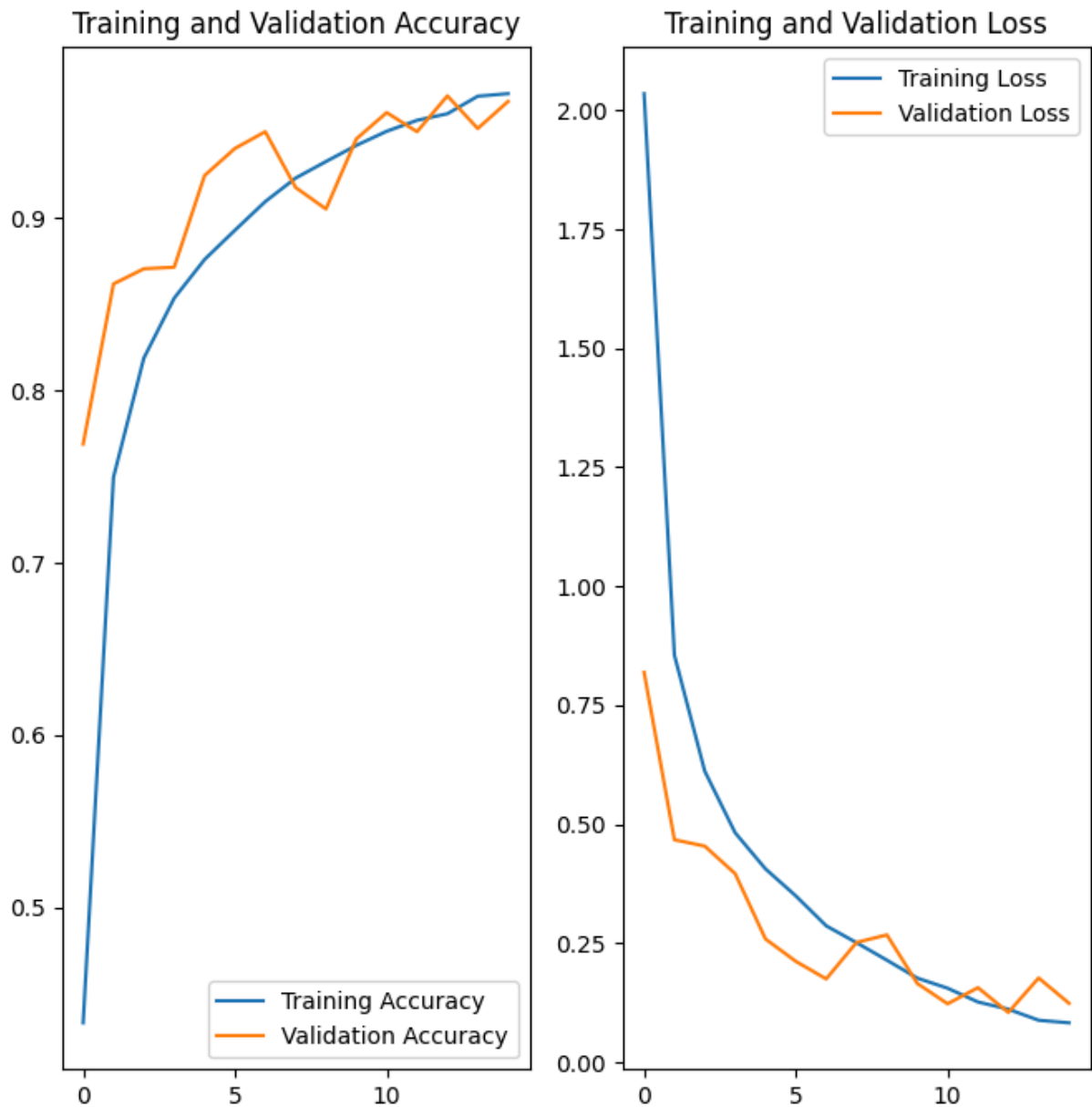
146/146			2s	11ms/step	
	precision	recall	f1-score	support	
I	0.94	0.97	0.95	75	
Ё	0.74	0.76	0.75	104	
A	0.88	0.80	0.84	141	
Б	0.82	0.87	0.85	134	
B	0.73	0.93	0.82	147	
Г	0.91	0.90	0.91	128	
Д	0.79	0.79	0.79	140	
E	0.83	0.69	0.76	173	
Ж	0.97	0.81	0.88	139	
З	0.92	0.92	0.92	130	
И	0.77	0.86	0.81	135	
Й	0.90	0.86	0.88	140	
K	0.94	0.88	0.91	138	
Л	0.82	0.97	0.89	130	
M	0.86	0.86	0.86	141	
H	0.82	0.91	0.86	153	
O	0.90	0.91	0.90	140	
П	0.82	0.88	0.85	143	
P	0.98	0.82	0.89	148	
C	0.98	0.90	0.94	154	
T	0.91	0.84	0.87	137	
У	0.94	0.98	0.96	166	
Ф	0.85	0.88	0.86	139	
X	0.95	0.87	0.91	146	
Ц	0.89	0.86	0.87	135	
Ч	0.94	0.85	0.89	140	
Ш	0.84	0.82	0.83	130	
Щ	0.87	0.80	0.83	134	
Ъ	0.90	0.87	0.89	138	
Ы	0.86	0.90	0.88	125	
Ь	0.89	0.94	0.91	129	
Э	0.96	0.93	0.95	138	
Ю	0.91	0.93	0.92	139	
Я	0.79	0.97	0.87	132	
accuracy			0.87	4661	
macro avg	0.88	0.87	0.87	4661	
weighted avg	0.88	0.87	0.87	4661	



```
In [12]: plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(
    epochs_range,
    results_main_model.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    epochs_range,
    results_main_model.history["val_accuracy"],
    label="Validation Accuracy",
)
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(
    epochs_range,
    results_main_model.history["loss"],
    label="Training Loss"
)
```

```
plt.plot(
    epochs_range,
    results_main_model.history["val_loss"],
    label="Validation Loss",
)
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```



```
In [13]: # Evaluate the model on the test dataset
test_loss, test_acc = model_main.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.2%}")
print(f"Test loss: {test_loss:.2%}")

# Compare to validation metrics (from the last epoch)
print(f"Validation accuracy: {results_main_model.history['val_accuracy'][-1]:.2%}")
print(f"Validation loss: {results_main_model.history['val_loss'][-1]:.2%}")

print("Difference")
```

```
print(f"Difference in accuracy: {results_main_model.history['val_accuracy'][-1] - test_accuracy}")
print(f"Difference in loss: {results_main_model.history['val_loss'][-1] - test_loss}")
```

146/146 ————— 2s 12ms/step - accuracy: 0.8726 - loss: 0.5061

Test accuracy: 87.26%

Test loss: 50.61%

Validation accuracy: 96.76%

Validation loss: 12.39%

Difference

Difference in accuracy: 9.51%

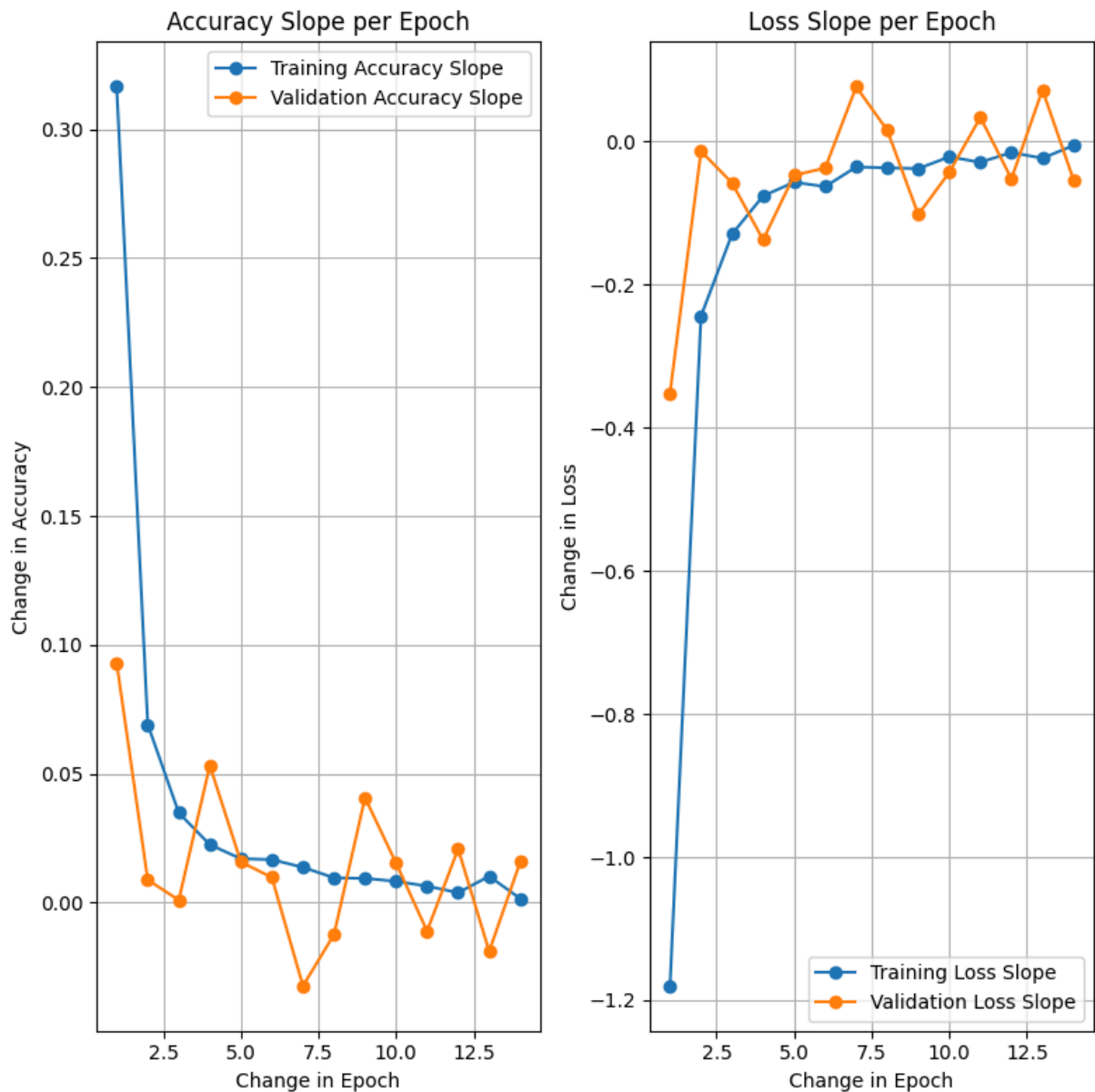
Difference in loss: -38.21%

```
In [14]: # Calculate slopes (differences between consecutive epochs)
acc_slope = np.diff(results_main_model.history["accuracy"])
val_acc_slope = np.diff(results_main_model.history["val_accuracy"])
loss_slope = np.diff(results_main_model.history["loss"])
val_loss_slope = np.diff(results_main_model.history["val_loss"])

# Plot the slopes
plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs), acc_slope, label="Training Accuracy Slope", marker="o")
plt.plot(range(1, epochs), val_acc_slope, label="Validation Accuracy Slope", marker="o")
plt.legend()
plt.title("Accuracy Slope per Epoch")
plt.xlabel("Change in Epoch")
plt.ylabel("Change in Accuracy")
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(1, epochs), loss_slope, label="Training Loss Slope", marker="o")
plt.plot(range(1, epochs), val_loss_slope, label="Validation Loss Slope", marker="o")
plt.legend()
plt.title("Loss Slope per Epoch")
plt.xlabel("Change in Epoch")
plt.ylabel("Change in Loss")
plt.grid(True)

plt.tight_layout()
plt.show()
```



Adding a Dropout Layer to improve the model's robustness

```
In [18]: model_with_dropout = tf.keras.Sequential(
    [
        tf.keras.layers.Rescaling(1.0 / 255),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(num_classes),
    ]
)
```

```
    ]  
)  
  
model_with_dropout.compile(  
    optimizer="adam",  
    loss=tf.keras.losses.CategoricalCrossentropy(from_logits=True),  
    metrics=["accuracy"],  
)
```

```
In [19]: results_dropout_model = model_with_dropout.fit(  
    training_dataset,  
    validation_data=validation_dataset,  
    epochs=epochs,  
    batch_size=32,  
    verbose=1  
)
```


Epoch 1/15
271/271  **12s** 26ms/step - accuracy: 0.2159 - loss: 2.7732
 - val_accuracy: 0.6445 - val_loss: 1.3234

Epoch 2/15
271/271  **3s** 9ms/step - accuracy: 0.5574 - loss: 1.4669 -
 val_accuracy: 0.8350 - val_loss: 0.6854

Epoch 3/15
271/271  **3s** 10ms/step - accuracy: 0.6720 - loss: 1.0742
 - val_accuracy: 0.8724 - val_loss: 0.4873

Epoch 4/15
271/271  **3s** 10ms/step - accuracy: 0.7257 - loss: 0.8640
 - val_accuracy: 0.8840 - val_loss: 0.3873

Epoch 5/15
271/271  **3s** 9ms/step - accuracy: 0.7720 - loss: 0.7357 -
 val_accuracy: 0.9015 - val_loss: 0.3518

Epoch 6/15
271/271  **2s** 9ms/step - accuracy: 0.7970 - loss: 0.6434 -
 val_accuracy: 0.9297 - val_loss: 0.2349

Epoch 7/15
271/271  **2s** 9ms/step - accuracy: 0.8103 - loss: 0.5866 -
 val_accuracy: 0.9307 - val_loss: 0.2446

Epoch 8/15
271/271  **3s** 10ms/step - accuracy: 0.8287 - loss: 0.5424
 - val_accuracy: 0.9565 - val_loss: 0.1642

Epoch 9/15
271/271  **4s** 13ms/step - accuracy: 0.8470 - loss: 0.4845
 - val_accuracy: 0.9616 - val_loss: 0.1520

Epoch 10/15
271/271  **3s** 10ms/step - accuracy: 0.8525 - loss: 0.4522
 - val_accuracy: 0.9538 - val_loss: 0.1622

Epoch 11/15
271/271  **3s** 10ms/step - accuracy: 0.8598 - loss: 0.4262
 - val_accuracy: 0.9561 - val_loss: 0.1529

Epoch 12/15
271/271  **2s** 9ms/step - accuracy: 0.8714 - loss: 0.3950 -
 val_accuracy: 0.9515 - val_loss: 0.1448

Epoch 13/15
271/271  **3s** 9ms/step - accuracy: 0.8767 - loss: 0.3767 -
 val_accuracy: 0.9639 - val_loss: 0.1267

Epoch 14/15
271/271  **3s** 10ms/step - accuracy: 0.8803 - loss: 0.3497
 - val_accuracy: 0.9612 - val_loss: 0.1247

Epoch 15/15
271/271  **3s** 11ms/step - accuracy: 0.8929 - loss: 0.3278
 - val_accuracy: 0.9570 - val_loss: 0.1315

```
In [17]: model_with_dropout.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Par
rescaling_1 (Rescaling)	(None, 32, 32, 4)	
conv2d_3 (Conv2D)	(None, 30, 30, 32)	1
max_pooling2d_3 (MaxPooling2D)	(None, 15, 15, 32)	
conv2d_4 (Conv2D)	(None, 13, 13, 32)	9
max_pooling2d_4 (MaxPooling2D)	(None, 6, 6, 32)	
conv2d_5 (Conv2D)	(None, 4, 4, 32)	9
max_pooling2d_5 (MaxPooling2D)	(None, 2, 2, 32)	
flatten_1 (Flatten)	(None, 128)	
dense_2 (Dense)	(None, 128)	16
dropout (Dropout)	(None, 128)	
dense_3 (Dense)	(None, 34)	4

Total params: 121,736 (475.54 KB)

Trainable params: 40,578 (158.51 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 81,158 (317.03 KB)

```
In [18]: # Evaluate the model on the test dataset
dropout_test_loss, dropout_test_acc = model_with_dropout.evaluate(test_data_loader)
print(f"Dropout Test accuracy: {dropout_test_acc:.2%}")
print(f"Dropout Test loss: {dropout_test_loss:.2%}")

# Compare to validation metrics (from the last epoch)
print(f"Dropout Validation accuracy: {results_dropout_model.history['val_acc'][-1]:.2%}")
print(f"Dropout Validation loss: {results_dropout_model.history['val_loss'][-1]:.2%}")

print(f"Dropout Difference in accuracy: {results_dropout_model.history['val_acc'][-1] - results_dropout_model.history['val_acc'][-2]:.2%}")
print(f"Dropout Difference in loss: {results_dropout_model.history['val_loss'][-1] - results_dropout_model.history['val_loss'][-2]:.2%}")
```

146/146 ————— 2s 11ms/step - accuracy: 0.8921 - loss: 0.3517

Dropout Test accuracy: 89.21%

Dropout Test loss: 35.17%

Dropout Validation accuracy: 95.65%

Dropout Validation loss: 14.06%

Dropout Difference in accuracy: 6.45%

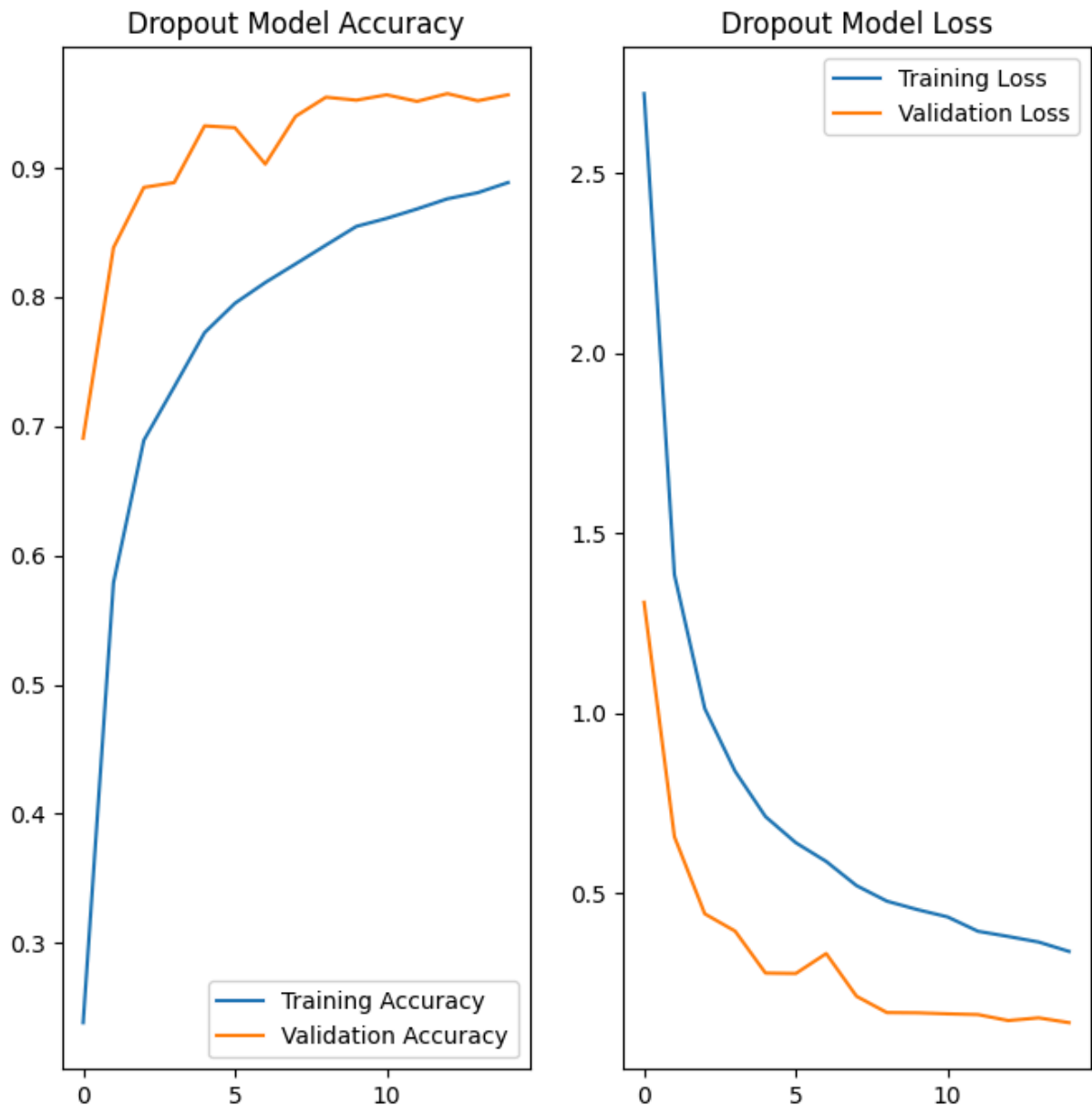
Dropout Difference in loss: -21.11%

```
In [19]: epochs_range = range(epochs)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, results_dropout_model.history["accuracy"], label="Training accuracy")
```

```
plt.plot(
    epochs_range, results_dropout_model.history["val_accuracy"], label="Vali
)
plt.legend(loc="lower right")
plt.title("Dropout Model Accuracy")

plt.subplot(1, 2, 2)
plt.plot(epochs_range, results_dropout_model.history["loss"], label="Trainin
plt.plot(epochs_range, results_dropout_model.history["val_loss"], label="Val
plt.legend(loc="upper right")
plt.title("Dropout Model Loss")
plt.show()
```



Let's see how accuracy varies in testing using Dropout

```
In [20]: epochs_range = range(epochs)

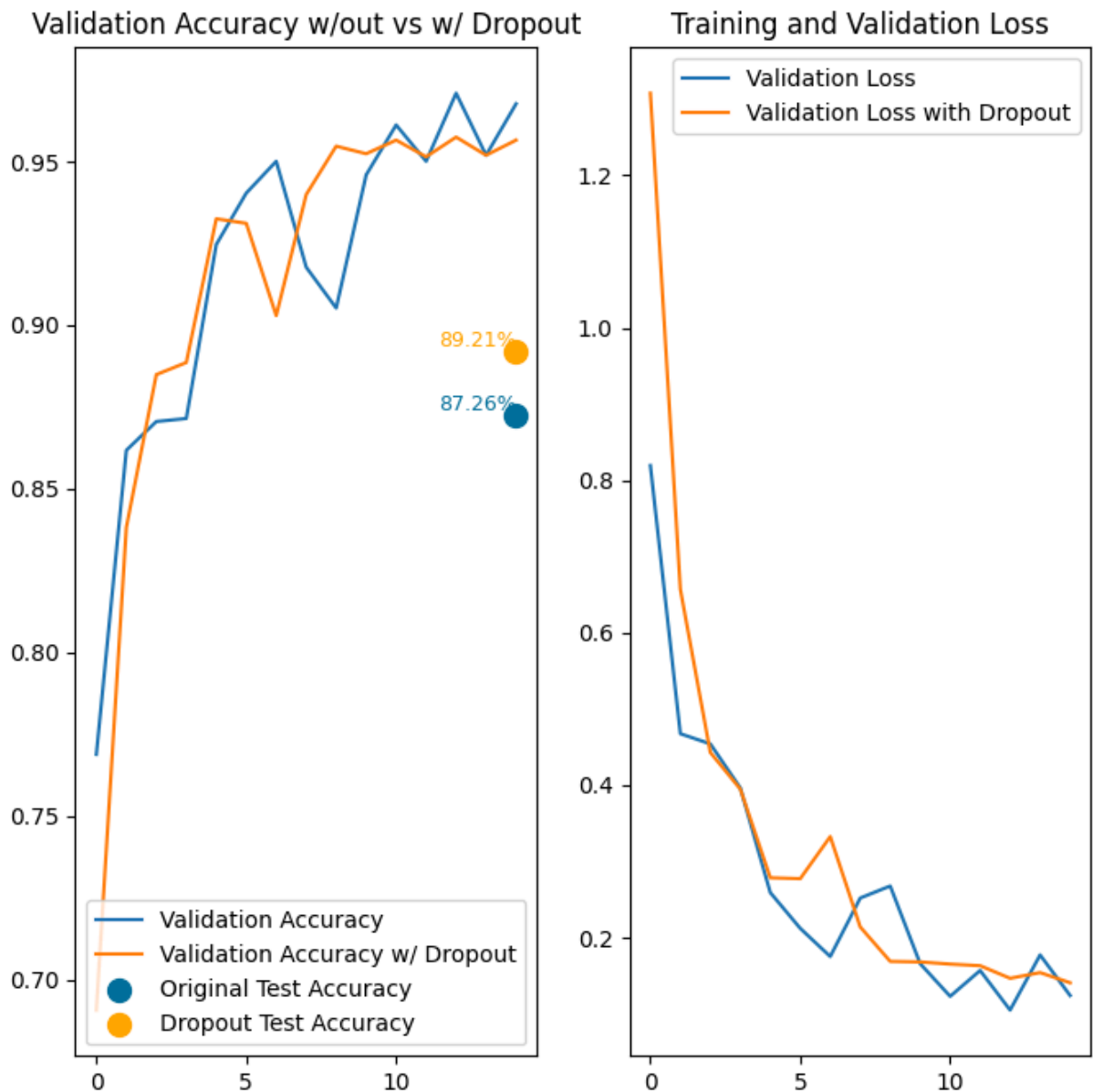
plt.figure(figsize=(8, 8))
```

```

plt.subplot(1, 2, 1)
plt.plot(epochs_range, results_main_model.history["val_accuracy"], label="Val")
plt.plot(
    epochs_range, results_dropout_model.history["val_accuracy"], label="Vali
)
plt.scatter(
    epochs - 1,
    test_acc,
    color="xkcd:ocean blue",
    s=100,
    zorder=5,
    label="Original Test Accuracy",
)
plt.text(
    epochs - 1,
    test_acc,
    f"{test_acc:.2%}",
    color="xkcd:ocean blue",
    fontsize=9,
    va="bottom",
    ha="right",
)
plt.scatter(
    epochs - 1,
    dropout_test_acc,
    color="orange",
    s=100,
    zorder=5,
    label="Dropout Test Accuracy",
)
plt.text(
    epochs - 1,
    dropout_test_acc,
    f"{dropout_test_acc:.2%}",
    color="orange",
    fontsize=9,
    va="bottom",
    ha="right",
)
plt.legend(loc="lower right")
plt.title("Validation Accuracy w/out vs w/ Dropout")

plt.subplot(1, 2, 2)
plt.plot(epochs_range, results_main_model.history["val_loss"], label="Valida")
plt.plot(epochs_range, results_dropout_model.history["val_loss"], label="Val")
plt.legend(loc="upper right")
plt.title("Training and Validation Loss")
plt.show()

```



```
In [21]: # Compute predictions and true labels
y_pred_probs = model_with_dropout.predict(test_dataset)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.concatenate(
    [np.argmax(labels_batch, axis=1) for _, labels_batch in test_dataset], a
)

# Confusion matrix and classification report
cm = confusion_matrix(y_true, y_pred)
print(classification_report(y_true, y_pred, target_names=class_names, zero_c

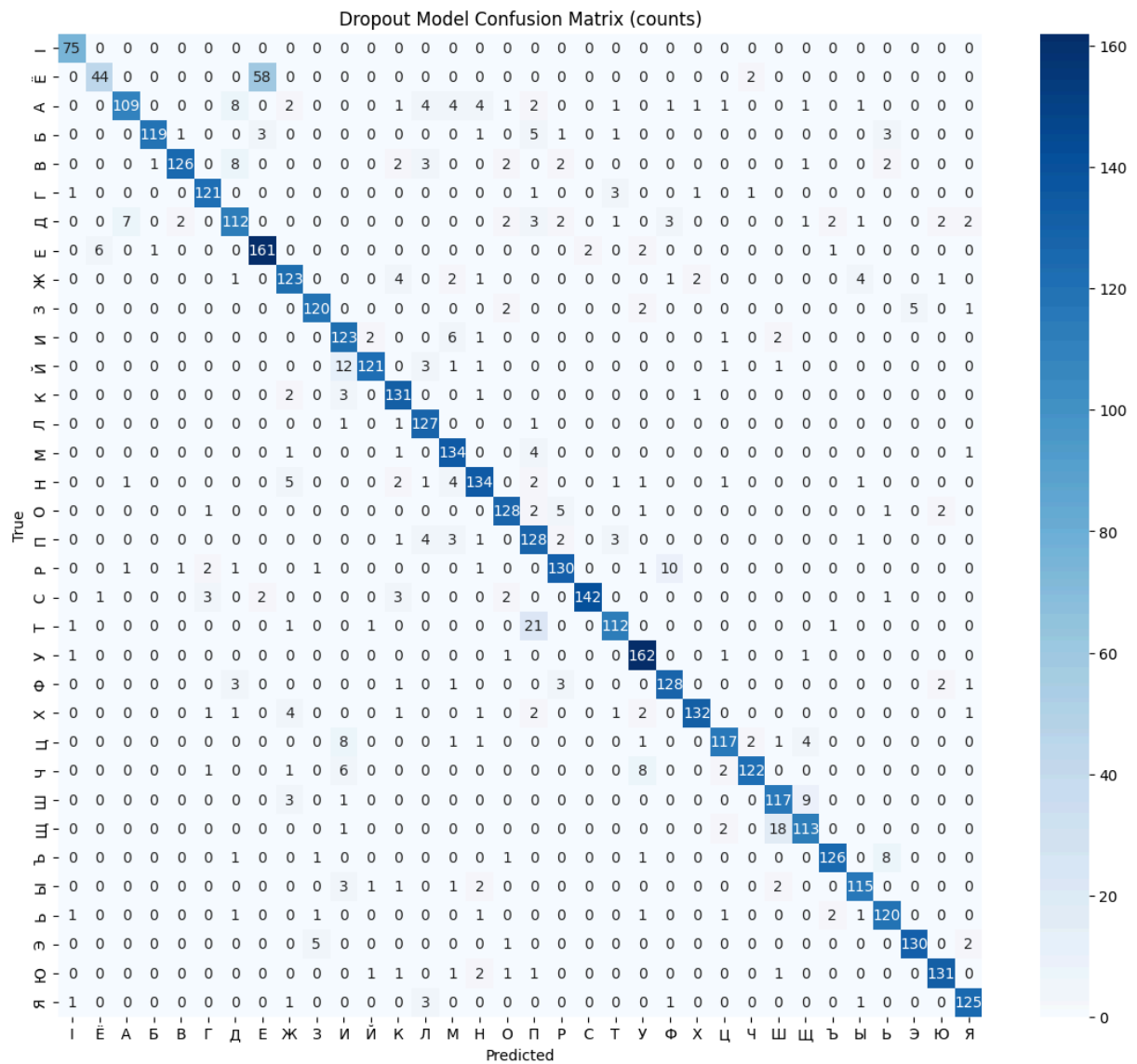
# Plot confusion matrix (raw counts)
plt.figure(figsize=(14, 12))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
```

```

    yticklabels=class_names,
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Dropout Model Confusion Matrix (counts)")
plt.show()

```

146/146			1s 8ms/step	
	precision	recall	f1-score	support
I	0.94	1.00	0.97	75
Ě	0.86	0.42	0.57	104
A	0.92	0.77	0.84	141
Б	0.98	0.89	0.93	134
B	0.97	0.86	0.91	147
Г	0.94	0.95	0.94	128
Д	0.82	0.80	0.81	140
E	0.72	0.93	0.81	173
Ж	0.86	0.88	0.87	139
З	0.94	0.92	0.93	130
И	0.78	0.91	0.84	135
Й	0.96	0.86	0.91	140
K	0.87	0.95	0.91	138
Л	0.88	0.98	0.92	130
M	0.85	0.95	0.90	141
H	0.88	0.88	0.88	153
O	0.91	0.91	0.91	140
П	0.74	0.90	0.81	143
P	0.90	0.88	0.89	148
C	0.99	0.92	0.95	154
T	0.91	0.82	0.86	137
У	0.89	0.98	0.93	166
Ф	0.89	0.92	0.90	139
X	0.96	0.90	0.93	146
Ц	0.92	0.87	0.89	135
Ч	0.96	0.87	0.91	140
Ш	0.82	0.90	0.86	130
Щ	0.87	0.84	0.86	134
Ъ	0.95	0.91	0.93	138
Ы	0.92	0.92	0.92	125
Ь	0.89	0.93	0.91	129
Э	0.96	0.94	0.95	138
Ю	0.95	0.94	0.95	139
Я	0.94	0.95	0.94	132
accuracy			0.89	4661
macro avg	0.90	0.89	0.89	4661
weighted avg	0.90	0.89	0.89	4661



What Dropout improved

Щ 0.90 0.72 0.80 134

Щ 0.93 0.83 0.88 134

we see that prediction improved on Щ and it doesn't get confused much from Ц or Ш

Dropout forced the model to not fixate on the vertical with horizontal features so much

Testing with different mini batches

16 batch

```
In [22]: model_with_dropout_16 = tf.keras.Sequential([
```

```

        tf.keras.layers.Rescaling(1.0 / 255),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(num_classes),
    ]
)

model_with_dropout_16.compile(
    optimizer="adam",
    loss=tf.keras.losses.CategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)

```

```

In [23]: results_dropout_model_16 = model_with_dropout_16.fit(
    training_dataset,
    validation_data=validation_dataset,
    epochs=epochs,
    batch_size=16,
    verbose=1
)

```


Epoch 1/15
271/271  **10s** 21ms/step - accuracy: 0.2384 - loss: 2.7292
 - val_accuracy: 0.6412 - val_loss: 1.3864

Epoch 2/15
271/271  **2s** 9ms/step - accuracy: 0.5712 - loss: 1.4254 -
 val_accuracy: 0.8326 - val_loss: 0.6455

Epoch 3/15
271/271  **2s** 9ms/step - accuracy: 0.6824 - loss: 1.0472 -
 val_accuracy: 0.8511 - val_loss: 0.5147

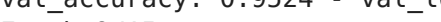
Epoch 4/15
271/271  **2s** 9ms/step - accuracy: 0.7353 - loss: 0.8567 -
 val_accuracy: 0.8835 - val_loss: 0.4064

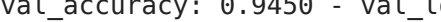
Epoch 5/15
271/271  **2s** 9ms/step - accuracy: 0.7662 - loss: 0.7485 -
 val_accuracy: 0.9108 - val_loss: 0.3344

Epoch 6/15
271/271  **2s** 9ms/step - accuracy: 0.7900 - loss: 0.6572 -
 val_accuracy: 0.9251 - val_loss: 0.2797

Epoch 7/15
271/271  **2s** 9ms/step - accuracy: 0.8162 - loss: 0.5883 -
 val_accuracy: 0.9404 - val_loss: 0.2346

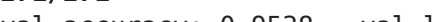
Epoch 8/15
271/271  **2s** 9ms/step - accuracy: 0.8340 - loss: 0.5197 -
 val_accuracy: 0.9524 - val_loss: 0.1907

Epoch 9/15
271/271  **2s** 9ms/step - accuracy: 0.8423 - loss: 0.4971 -
 val_accuracy: 0.9450 - val_loss: 0.2024

Epoch 10/15
271/271  **2s** 9ms/step - accuracy: 0.8533 - loss: 0.4510 -
 val_accuracy: 0.9704 - val_loss: 0.1284

Epoch 11/15
271/271  **2s** 9ms/step - accuracy: 0.8637 - loss: 0.4214 -
 val_accuracy: 0.9575 - val_loss: 0.1684

Epoch 12/15
271/271  **2s** 9ms/step - accuracy: 0.8708 - loss: 0.3964 -
 val_accuracy: 0.9538 - val_loss: 0.1607

Epoch 13/15
271/271  **2s** 8ms/step - accuracy: 0.8778 - loss: 0.3701 -
 val_accuracy: 0.9755 - val_loss: 0.1091

Epoch 14/15
271/271  **2s** 9ms/step - accuracy: 0.8819 - loss: 0.3591 -
 val_accuracy: 0.9496 - val_loss: 0.1581

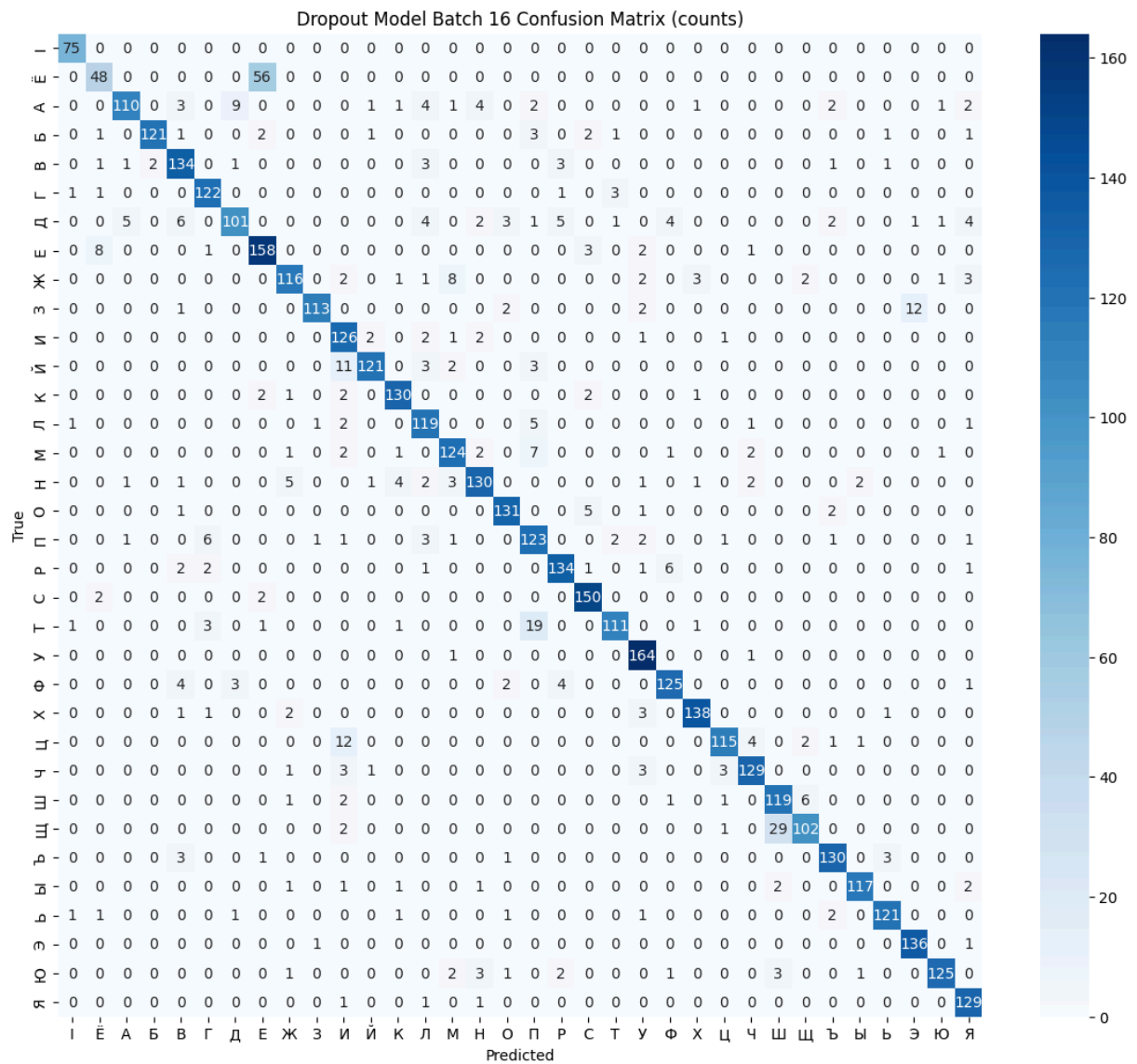
Epoch 15/15
271/271  **2s** 9ms/step - accuracy: 0.8901 - loss: 0.3273 -
 val_accuracy: 0.9630 - val_loss: 0.1278

```
In [24]: # Compute predictions and true labels
y_pred_probs = model_with_dropout_16.predict(test_dataset)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.concatenate(
    [np.argmax(labels_batch, axis=1) for _, labels_batch in test_dataset], a
)

# Confusion matrix and classification report
cm = confusion_matrix(y_true, y_pred)
print(classification_report(y_true, y_pred, target_names=class_names, zero_c
```

```
# Plot confusion matrix (raw counts)
plt.figure(figsize=(14, 12))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names,
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Dropout Model Batch 16 Confusion Matrix (counts)")
plt.show()
```

	precision	recall	f1-score	support
I	0.95	1.00	0.97	75
Ё	0.77	0.46	0.58	104
A	0.93	0.78	0.85	141
Б	0.98	0.90	0.94	134
B	0.85	0.91	0.88	147
Г	0.90	0.95	0.93	128
Д	0.88	0.72	0.79	140
E	0.71	0.91	0.80	173
Ж	0.90	0.83	0.87	139
З	0.97	0.87	0.92	130
И	0.75	0.93	0.83	135
Й	0.95	0.86	0.91	140
K	0.93	0.94	0.94	138
Л	0.83	0.92	0.87	130
М	0.87	0.88	0.87	141
Н	0.90	0.85	0.87	153
O	0.93	0.94	0.93	140
П	0.75	0.86	0.80	143
P	0.90	0.91	0.90	148
C	0.92	0.97	0.95	154
T	0.94	0.81	0.87	137
У	0.90	0.99	0.94	166
Ф	0.91	0.90	0.90	139
X	0.95	0.95	0.95	146
Ц	0.94	0.85	0.89	135
Ч	0.92	0.92	0.92	140
Ш	0.78	0.92	0.84	130
Щ	0.91	0.76	0.83	134
Ъ	0.92	0.94	0.93	138
Ы	0.97	0.94	0.95	125
Ь	0.95	0.94	0.95	129
Э	0.91	0.99	0.95	138
Ю	0.97	0.90	0.93	139
Я	0.88	0.98	0.93	132
accuracy			0.89	4661
macro avg	0.90	0.89	0.89	4661
weighted avg	0.89	0.89	0.89	4661



64 batch

```
In [25]: model_with_dropout_64 = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0 / 255),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(num_classes),
])

model_with_dropout_64.compile(
    optimizer="adam",
```

```
    loss=tf.keras.losses.CategoricalCrossentropy(from_logits=True),  
    metrics=["accuracy"],  
)
```

```
In [26]: results_dropout_model_64 = model_with_dropout_64.fit(  
    training_dataset,  
    validation_data=validation_dataset,  
    epochs=epochs,  
    batch_size=64,  
    verbose=1  
)
```

Epoch 1/15
271/271  **9s** 21ms/step - accuracy: 0.2339 - loss: 2.7484
 - val_accuracy: 0.5904 - val_loss: 1.4851

Epoch 2/15
271/271  **2s** 9ms/step - accuracy: 0.5710 - loss: 1.4616 -
 val_accuracy: 0.8003 - val_loss: 0.7200

Epoch 3/15
271/271  **2s** 9ms/step - accuracy: 0.6885 - loss: 1.0302 -
 val_accuracy: 0.8479 - val_loss: 0.5460

Epoch 4/15
271/271  **3s** 9ms/step - accuracy: 0.7428 - loss: 0.8299 -
 val_accuracy: 0.9034 - val_loss: 0.3743

Epoch 5/15
271/271  **2s** 9ms/step - accuracy: 0.7852 - loss: 0.7066 -
 val_accuracy: 0.9177 - val_loss: 0.2962

Epoch 6/15
271/271  **2s** 9ms/step - accuracy: 0.8003 - loss: 0.6279 -
 val_accuracy: 0.8974 - val_loss: 0.3494

Epoch 7/15
271/271  **2s** 9ms/step - accuracy: 0.8222 - loss: 0.5639 -
 val_accuracy: 0.9334 - val_loss: 0.2166

Epoch 8/15
271/271  **2s** 9ms/step - accuracy: 0.8325 - loss: 0.5222 -
 val_accuracy: 0.9431 - val_loss: 0.1924

Epoch 9/15
271/271  **2s** 9ms/step - accuracy: 0.8440 - loss: 0.4772 -
 val_accuracy: 0.9501 - val_loss: 0.1691

Epoch 10/15
271/271  **2s** 9ms/step - accuracy: 0.8573 - loss: 0.4407 -
 val_accuracy: 0.9612 - val_loss: 0.1548

Epoch 11/15
271/271  **3s** 9ms/step - accuracy: 0.8649 - loss: 0.4023 -
 val_accuracy: 0.9639 - val_loss: 0.1351

Epoch 12/15
271/271  **2s** 9ms/step - accuracy: 0.8715 - loss: 0.3946 -
 val_accuracy: 0.9621 - val_loss: 0.1293

Epoch 13/15
271/271  **2s** 9ms/step - accuracy: 0.8822 - loss: 0.3550 -
 val_accuracy: 0.9552 - val_loss: 0.1478

Epoch 14/15
271/271  **2s** 9ms/step - accuracy: 0.8896 - loss: 0.3325 -
 val_accuracy: 0.9552 - val_loss: 0.1458

Epoch 15/15
271/271  **2s** 9ms/step - accuracy: 0.8914 - loss: 0.3279 -
 val_accuracy: 0.9713 - val_loss: 0.1067

```
In [27]: last_ten_epochs = range(epochs - 10, epochs)
plt.figure(figsize=(8, 8))

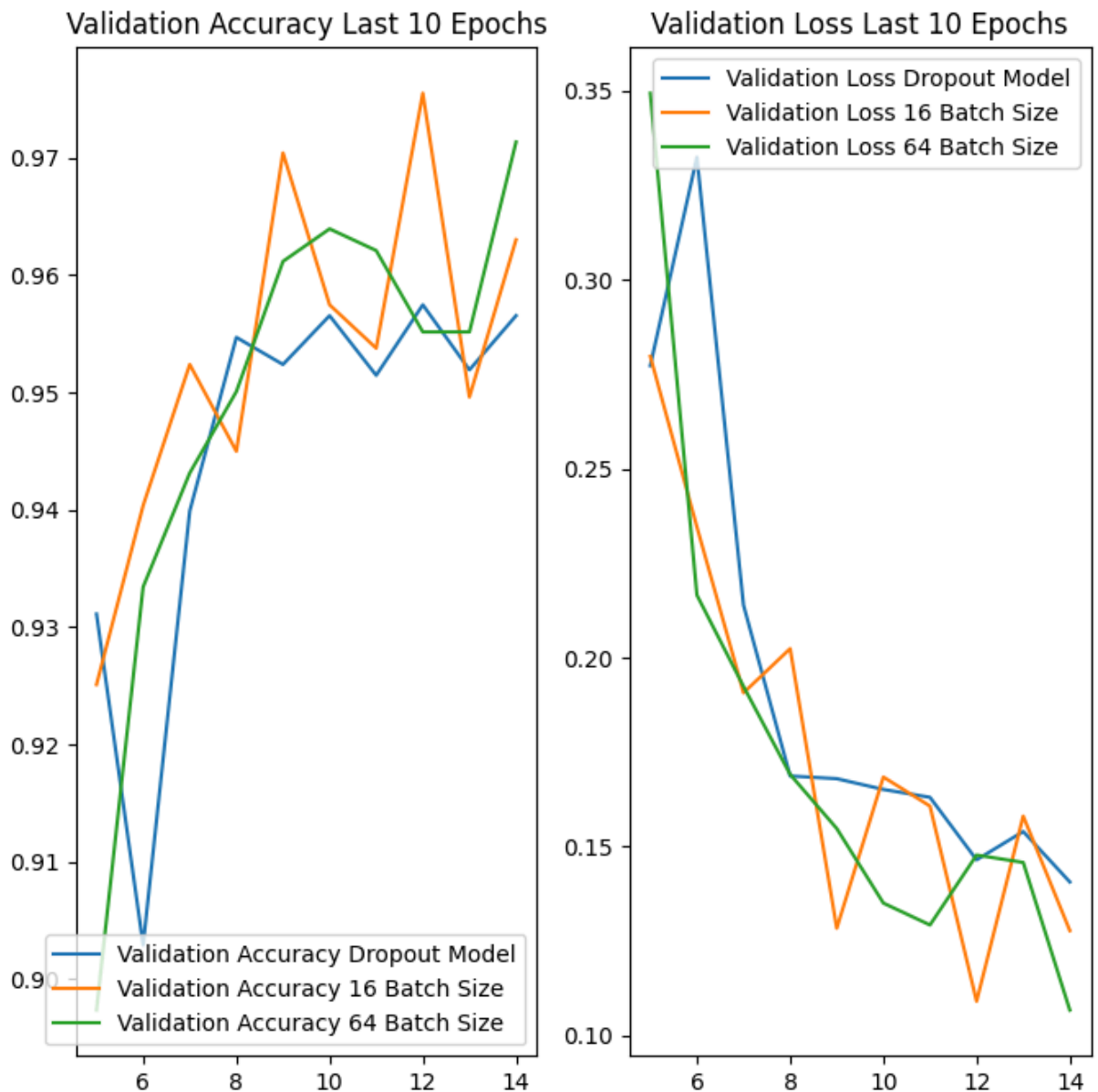
plt.subplot(1, 2, 1)
plt.plot(
    last_ten_epochs,
    results_dropout_model.history["val_accuracy"][-10:],
    label="Validation Accuracy Dropout Model",
)
plt.plot(
```

```

        last_ten_epochs,
        results_dropout_model_16.history["val_accuracy"][-10:],
        label="Validation Accuracy 16 Batch Size",
    )
plt.plot(
    last_ten_epochs,
    results_dropout_model_64.history["val_accuracy"][-10:],
    label="Validation Accuracy 64 Batch Size",
)
plt.legend(loc="lower right")
plt.title("Validation Accuracy Last 10 Epochs")

plt.subplot(1, 2, 2)
plt.plot(
    last_ten_epochs,
    results_dropout_model.history["val_loss"][-10:],
    label="Validation Loss Dropout Model",
)
plt.plot(
    last_ten_epochs,
    results_dropout_model_16.history["val_loss"][-10:],
    label="Validation Loss 16 Batch Size",
)
plt.plot(
    last_ten_epochs,
    results_dropout_model_64.history["val_loss"][-10:],
    label="Validation Loss 64 Batch Size",
)
plt.legend(loc="upper right")
plt.title("Validation Loss Last 10 Epochs")
plt.show()

```



```
In [28]: # Compute predictions and true labels
y_pred_probs = model_with_dropout_64.predict(test_dataset)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.concatenate(
    [np.argmax(labels_batch, axis=1) for _, labels_batch in test_dataset], axis=0
)

# Confusion matrix and classification report
cm = confusion_matrix(y_true, y_pred)
print(classification_report(y_true, y_pred, target_names=class_names, zero_division=0))

# Plot confusion matrix (raw counts)
plt.figure(figsize=(14, 12))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
```

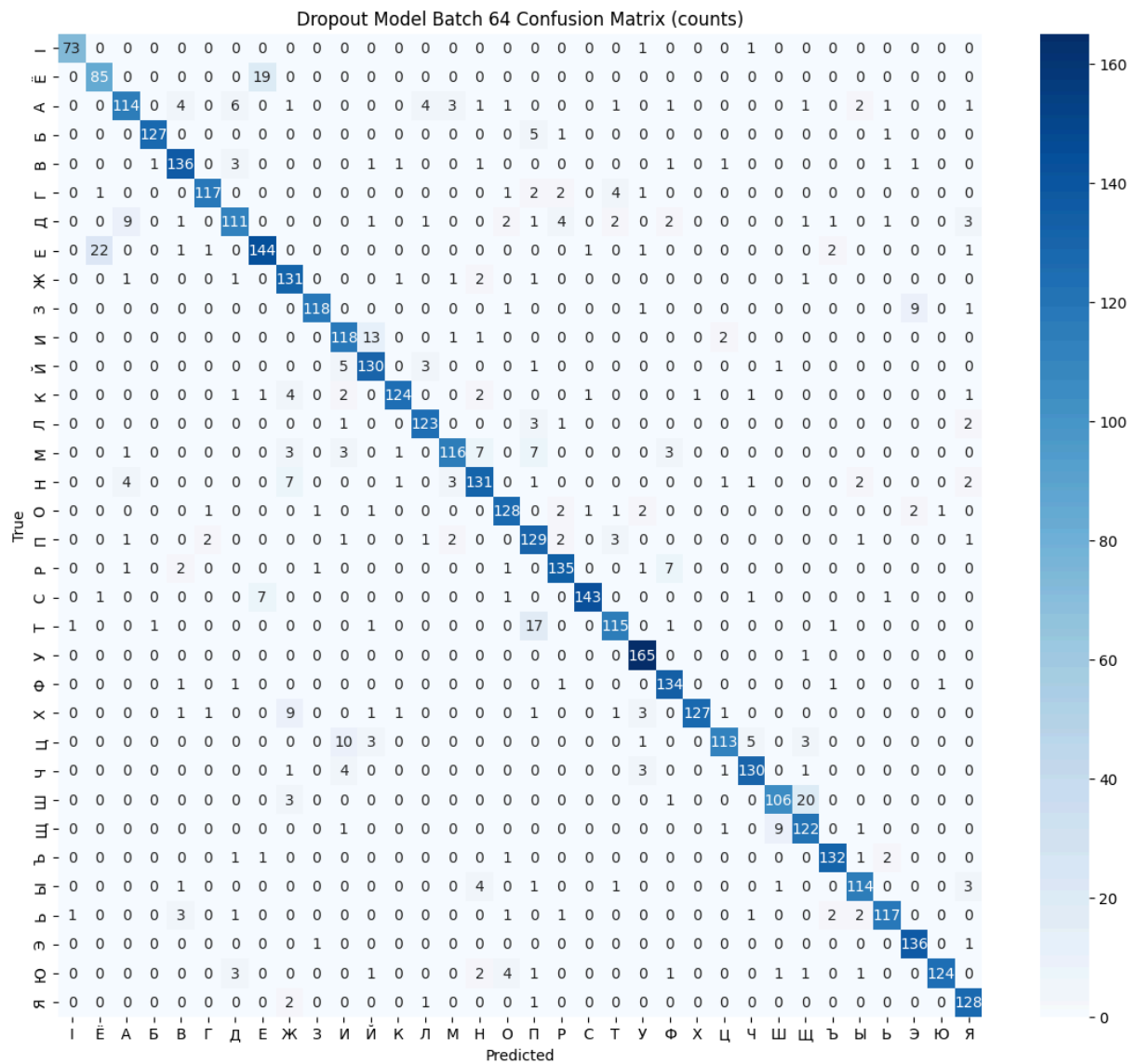


```

    yticklabels=class_names,
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Dropout Model Batch 64 Confusion Matrix (counts)")
plt.show()

```

146/146					1s 8ms/step
	precision	recall	f1-score	support	
I	0.97	0.97	0.97	75	
Ė	0.78	0.82	0.80	104	
A	0.87	0.81	0.84	141	
Б	0.98	0.95	0.97	134	
B	0.91	0.93	0.92	147	
Г	0.96	0.91	0.94	128	
Д	0.87	0.79	0.83	140	
E	0.84	0.83	0.83	173	
Ж	0.81	0.94	0.87	139	
З	0.98	0.91	0.94	130	
И	0.81	0.87	0.84	135	
Й	0.86	0.93	0.89	140	
K	0.96	0.90	0.93	138	
Л	0.92	0.95	0.94	130	
M	0.92	0.82	0.87	141	
H	0.87	0.86	0.86	153	
O	0.91	0.91	0.91	140	
П	0.75	0.90	0.82	143	
P	0.91	0.91	0.91	148	
C	0.98	0.93	0.95	154	
T	0.90	0.84	0.87	137	
У	0.92	0.99	0.96	166	
Ф	0.89	0.96	0.92	139	
X	0.99	0.87	0.93	146	
Ц	0.94	0.84	0.89	135	
Ч	0.93	0.93	0.93	140	
Ш	0.90	0.82	0.85	130	
Щ	0.81	0.91	0.86	134	
Ъ	0.95	0.96	0.95	138	
Ы	0.92	0.91	0.92	125	
Ь	0.94	0.91	0.92	129	
Э	0.92	0.99	0.95	138	
Ю	0.98	0.89	0.94	139	
Я	0.89	0.97	0.93	132	
accuracy			0.90	4661	
macro avg	0.90	0.90	0.90	4661	
weighted avg	0.90	0.90	0.90	4661	



Testing with mean squared error

```
In [ ]: model_with_dropout_mse = tf.keras.Sequential(
    [
        tf.keras.layers.Rescaling(1.0 / 255),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(num_classes, activation='softmax'), # MSE requ
    ]
)

model_with_dropout_mse.compile(
```

```
optimizer="adam",  
loss="mean_squared_error",  
metrics=["accuracy"],  
)
```

```
In [16]: results_dropout_model_mse = model_with_dropout_mse.fit(  
    training_dataset,  
    validation_data=validation_dataset,  
    epochs=epochs,  
    verbose=1  
)
```

Epoch 1/15

WARNING: All log messages before absl::InitializeLog() is called are written to STDERR

I0000 00:00:1782480015.574070 6969 service.cc:146] XLA service 0x7c6978005ca0 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:

I0000 00:00:1782480015.574105 6969 service.cc:154] StreamExecutor device (0): NVIDIA GeForce GTX 1050 Ti, Compute Capability 6.1

21/271 ————— **2s** 8ms/step - accuracy: 0.0318 - loss: 0.0285

I0000 00:00:1782480020.023025 6969 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.

271/271  **15s** 30ms/step - accuracy: 0.2384 - loss: 0.0254
 - val_accuracy: 0.6006 - val_loss: 0.0166
 Epoch 2/15
271/271  **2s** 9ms/step - accuracy: 0.5365 - loss: 0.0177 -
 val_accuracy: 0.7605 - val_loss: 0.0102
 Epoch 3/15
271/271  **3s** 9ms/step - accuracy: 0.6612 - loss: 0.0135 -
 val_accuracy: 0.8202 - val_loss: 0.0075
 Epoch 4/15
271/271  **2s** 9ms/step - accuracy: 0.7198 - loss: 0.0114 -
 val_accuracy: 0.8608 - val_loss: 0.0058
 Epoch 5/15
271/271  **3s** 9ms/step - accuracy: 0.7553 - loss: 0.0100 -
 val_accuracy: 0.8988 - val_loss: 0.0045
 Epoch 6/15
271/271  **3s** 9ms/step - accuracy: 0.7835 - loss: 0.0090 -
 val_accuracy: 0.8849 - val_loss: 0.0049
 Epoch 7/15
271/271  **2s** 9ms/step - accuracy: 0.8073 - loss: 0.0081 -
 val_accuracy: 0.9163 - val_loss: 0.0036
 Epoch 8/15
271/271  **2s** 9ms/step - accuracy: 0.8189 - loss: 0.0077 -
 val_accuracy: 0.9316 - val_loss: 0.0030
 Epoch 9/15
271/271  **2s** 9ms/step - accuracy: 0.8341 - loss: 0.0071 -
 val_accuracy: 0.9431 - val_loss: 0.0025
 Epoch 10/15
271/271  **2s** 9ms/step - accuracy: 0.8470 - loss: 0.0065 -
 val_accuracy: 0.9390 - val_loss: 0.0027
 Epoch 11/15
271/271  **3s** 9ms/step - accuracy: 0.8544 - loss: 0.0062 -
 val_accuracy: 0.9302 - val_loss: 0.0030
 Epoch 12/15
271/271  **2s** 9ms/step - accuracy: 0.8658 - loss: 0.0058 -
 val_accuracy: 0.9390 - val_loss: 0.0027
 Epoch 13/15
271/271  **2s** 9ms/step - accuracy: 0.8700 - loss: 0.0056 -
 val_accuracy: 0.9542 - val_loss: 0.0021
 Epoch 14/15
271/271  **2s** 9ms/step - accuracy: 0.8790 - loss: 0.0053 -
 val_accuracy: 0.9593 - val_loss: 0.0018
 Epoch 15/15
271/271  **2s** 9ms/step - accuracy: 0.8826 - loss: 0.0050 -
 val_accuracy: 0.9427 - val_loss: 0.0025

```

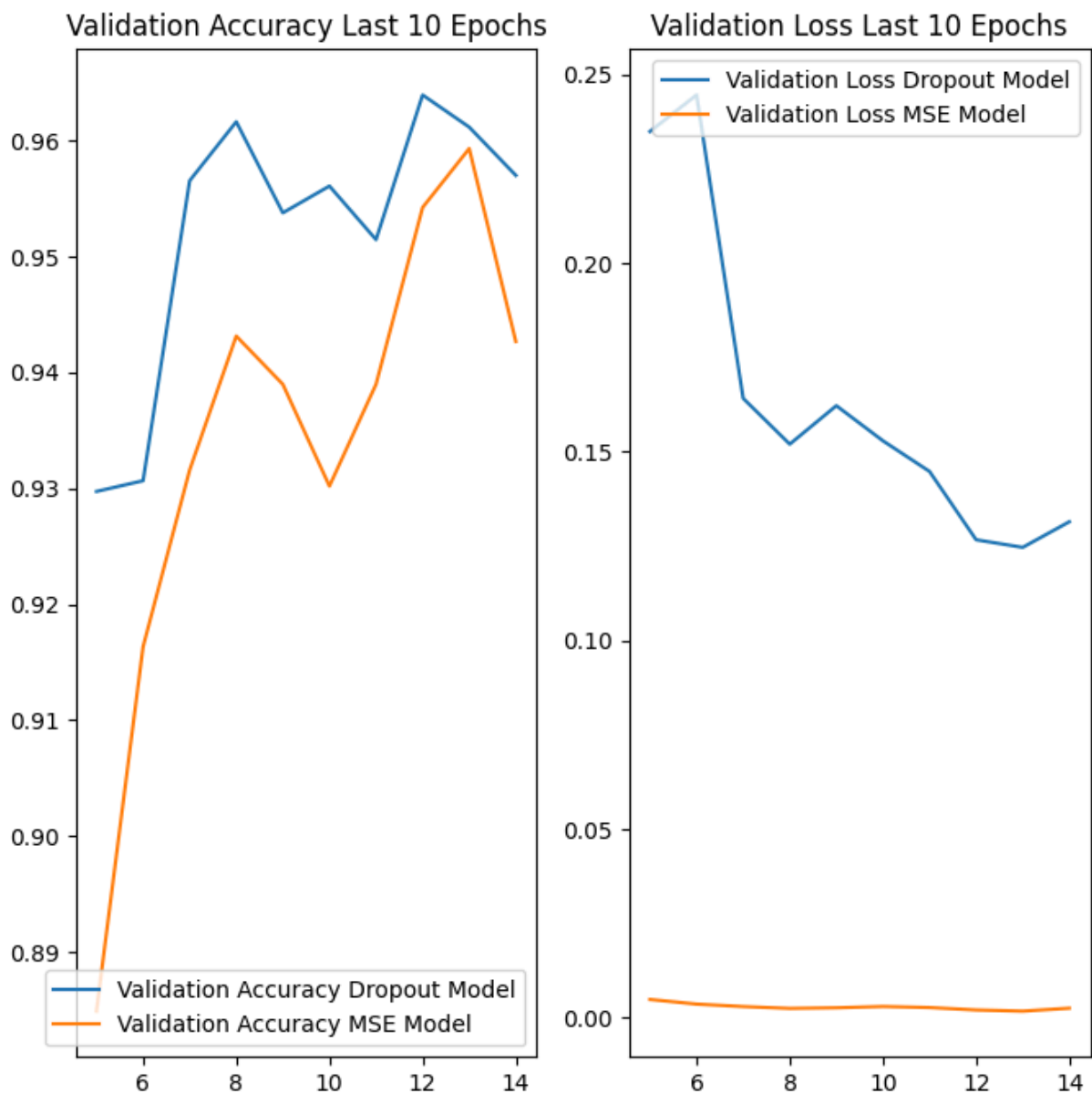
In [21]: last_ten_epochs = range(epochs - 10, epochs)
plt.figure(figsize=(8, 8))

plt.subplot(1, 2, 1)
plt.plot(
    last_ten_epochs,
    results_dropout_model.history["val_accuracy"][-10:],
    label="Validation Accuracy Dropout Model",
)
plt.plot(
    last_ten_epochs,

```

```
    results_dropout_model_mse.history["val_accuracy"][-10:],
    label="Validation Accuracy MSE Model",
)
plt.legend(loc="lower right")
plt.title("Validation Accuracy Last 10 Epochs")

plt.subplot(1, 2, 2)
plt.plot(
    last_ten_epochs,
    results_dropout_model_mse.history["val_loss"][-10:],
    label="Validation Loss Dropout Model",
)
plt.plot(
    last_ten_epochs,
    results_dropout_model_mse.history["val_loss"][-10:],
    label="Validation Loss MSE Model",
)
plt.legend(loc="upper right")
plt.title("Validation Loss Last 10 Epochs")
plt.show()
```



```
In [32]: # Compute predictions and true labels
y_pred_probs = model_with_dropout_mse.predict(test_dataset)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.concatenate(
    [np.argmax(labels_batch, axis=1) for _, labels_batch in test_dataset], axis=0)

# Confusion matrix and classification report
cm = confusion_matrix(y_true, y_pred)
print(classification_report(y_true, y_pred, target_names=class_names, zero_division=0))

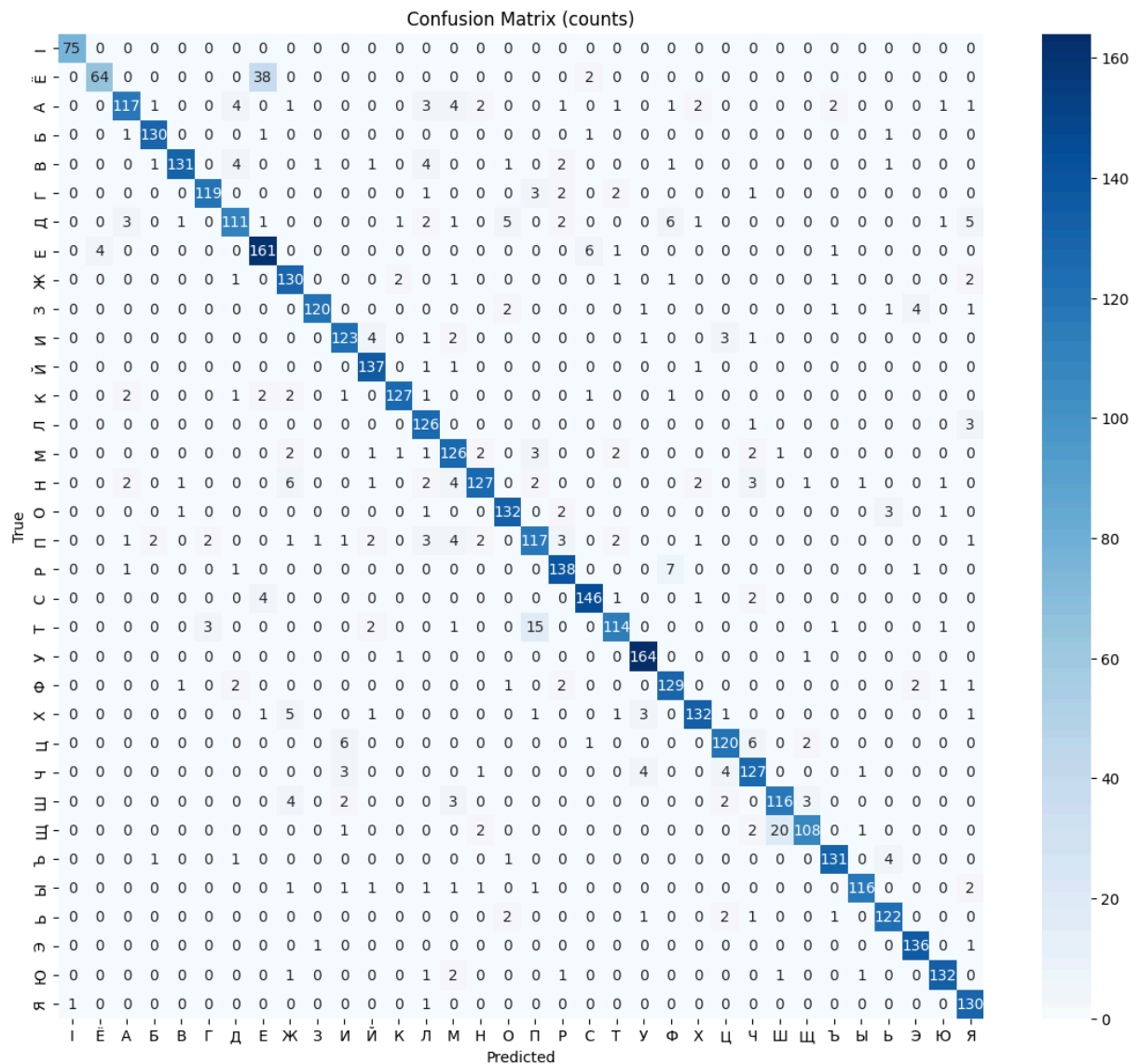
# Plot confusion matrix (raw counts)
plt.figure(figsize=(14, 12))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
```

```

    yticklabels=class_names,
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix (counts)")
plt.show()

```

146/146					1s 8ms/step
	precision	recall	f1-score	support	
I	0.99	1.00	0.99	75	
Ě	0.94	0.62	0.74	104	
A	0.92	0.83	0.87	141	
Б	0.96	0.97	0.97	134	
B	0.97	0.89	0.93	147	
Г	0.96	0.93	0.94	128	
Д	0.89	0.79	0.84	140	
E	0.77	0.93	0.85	173	
Ж	0.85	0.94	0.89	139	
З	0.98	0.92	0.95	130	
И	0.89	0.91	0.90	135	
Й	0.91	0.98	0.94	140	
K	0.96	0.92	0.94	138	
Л	0.85	0.97	0.90	130	
M	0.84	0.89	0.87	141	
H	0.93	0.83	0.88	153	
O	0.92	0.94	0.93	140	
П	0.82	0.82	0.82	143	
P	0.90	0.93	0.92	148	
C	0.93	0.95	0.94	154	
T	0.91	0.83	0.87	137	
У	0.94	0.99	0.96	166	
Ф	0.88	0.93	0.91	139	
X	0.94	0.90	0.92	146	
Ц	0.91	0.89	0.90	135	
Ч	0.87	0.91	0.89	140	
Ш	0.84	0.89	0.87	130	
Щ	0.94	0.81	0.87	134	
Ъ	0.95	0.95	0.95	138	
Ы	0.97	0.93	0.95	125	
Ь	0.92	0.95	0.93	129	
Э	0.95	0.99	0.97	138	
Ю	0.96	0.95	0.95	139	
Я	0.88	0.98	0.93	132	
accuracy			0.91	4661	
macro avg	0.91	0.91	0.91	4661	
weighted avg	0.91	0.91	0.91	4661	



Trying different final activation functions

So far we have been using raw logits as the output. We'll try activation functions on the output layer

Softmax activation

```
In [33]: model_with_dropout_softmax = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1.0 / 255),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
```



```
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(num_classes, activation="softmax"),
    ]
)

model_with_dropout_softmax.compile(
    optimizer="adam",
    loss=tf.keras.losses.CategoricalCrossentropy(from_logits=False),
    metrics=["accuracy"],
)
```

```
In [34]: results_dropout_model_softmax = model_with_dropout_softmax.fit(
    training_dataset, validation_data=validation_dataset, epochs=epochs, vert
)
```

Epoch 1/15
271/271  **11s** 24ms/step - accuracy: 0.2408 - loss: 2.7611
 - val_accuracy: 0.6343 - val_loss: 1.4927

Epoch 2/15
271/271  **2s** 9ms/step - accuracy: 0.5680 - loss: 1.4511 -
 val_accuracy: 0.8266 - val_loss: 0.6859

Epoch 3/15
271/271  **2s** 9ms/step - accuracy: 0.6799 - loss: 1.0485 -
 val_accuracy: 0.8433 - val_loss: 0.5562

Epoch 4/15
271/271  **2s** 9ms/step - accuracy: 0.7397 - loss: 0.8413 -
 val_accuracy: 0.8941 - val_loss: 0.3788

Epoch 5/15
271/271  **2s** 9ms/step - accuracy: 0.7701 - loss: 0.7267 -
 val_accuracy: 0.9182 - val_loss: 0.3388

Epoch 6/15
271/271  **2s** 9ms/step - accuracy: 0.7963 - loss: 0.6413 -
 val_accuracy: 0.9325 - val_loss: 0.2576

Epoch 7/15
271/271  **2s** 9ms/step - accuracy: 0.8172 - loss: 0.5890 -
 val_accuracy: 0.9237 - val_loss: 0.2729

Epoch 8/15
271/271  **2s** 9ms/step - accuracy: 0.8328 - loss: 0.5359 -
 val_accuracy: 0.9348 - val_loss: 0.2179

Epoch 9/15
271/271  **2s** 9ms/step - accuracy: 0.8439 - loss: 0.4838 -
 val_accuracy: 0.9505 - val_loss: 0.1768

Epoch 10/15
271/271  **2s** 9ms/step - accuracy: 0.8571 - loss: 0.4393 -
 val_accuracy: 0.9445 - val_loss: 0.1962

Epoch 11/15
271/271  **2s** 9ms/step - accuracy: 0.8689 - loss: 0.4159 -
 val_accuracy: 0.9515 - val_loss: 0.1693

Epoch 12/15
271/271  **2s** 9ms/step - accuracy: 0.8711 - loss: 0.3995 -
 val_accuracy: 0.9468 - val_loss: 0.1837

Epoch 13/15
271/271  **2s** 9ms/step - accuracy: 0.8794 - loss: 0.3678 -
 val_accuracy: 0.9584 - val_loss: 0.1538

Epoch 14/15
271/271  **2s** 9ms/step - accuracy: 0.8786 - loss: 0.3573 -
 val_accuracy: 0.9575 - val_loss: 0.1385

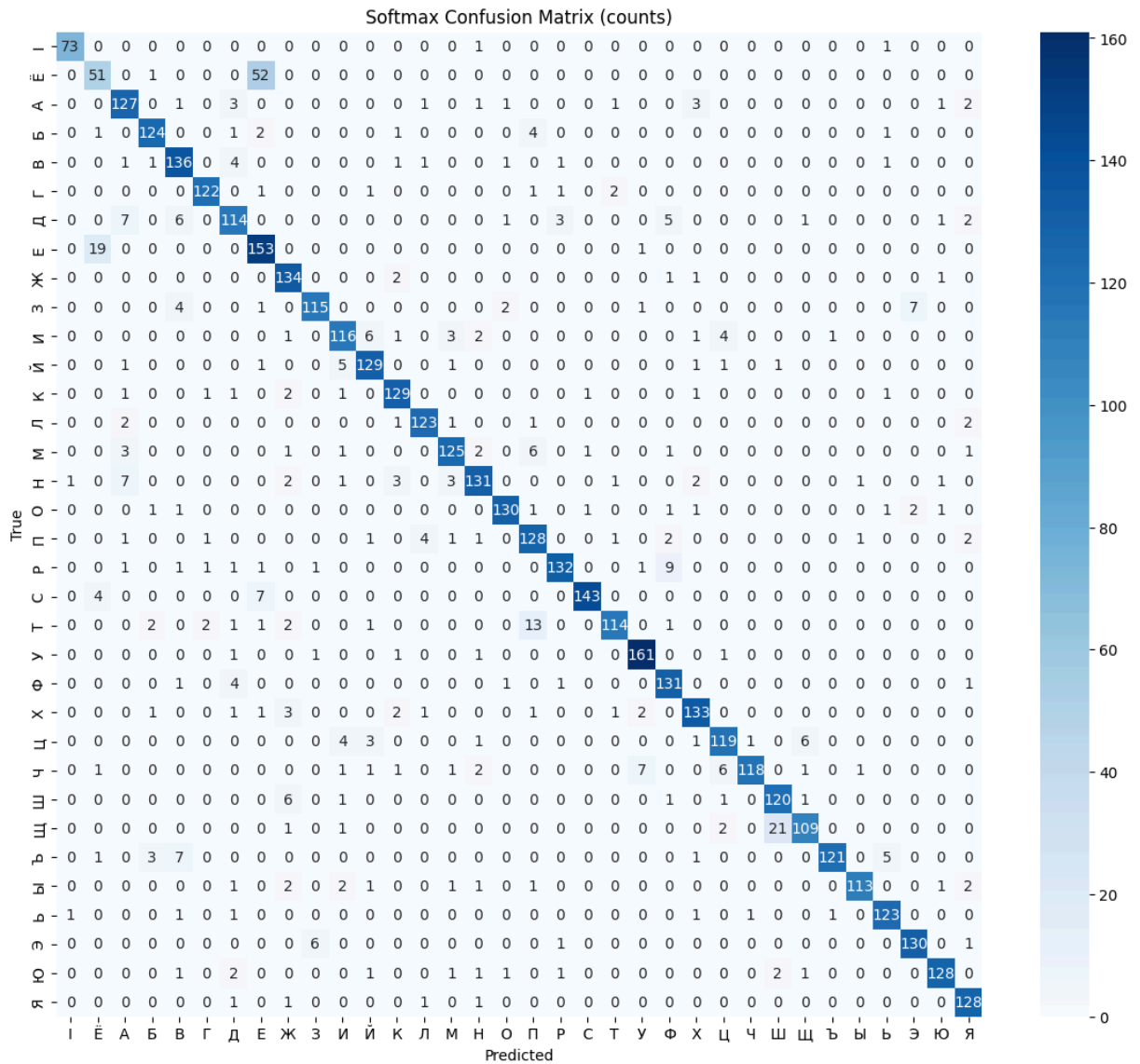
Epoch 15/15
271/271  **2s** 9ms/step - accuracy: 0.8905 - loss: 0.3354 -
 val_accuracy: 0.9556 - val_loss: 0.1612

```
In [35]: # Compute predictions and true labels
y_pred_probs = model_with_dropout_softmax.predict(test_dataset)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.concatenate(
    [np.argmax(labels_batch, axis=1) for _, labels_batch in test_dataset], a
)

# Confusion matrix and classification report
cm = confusion_matrix(y_true, y_pred)
print(classification_report(y_true, y_pred, target_names=class_names, zero_c
```

```
# Plot confusion matrix (raw counts)
plt.figure(figsize=(14, 12))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names,
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Softmax Confusion Matrix (counts)")
plt.show()
```

	precision	recall	f1-score	support
I	0.97	0.97	0.97	75
Ě	0.66	0.49	0.56	104
A	0.84	0.90	0.87	141
Б	0.93	0.93	0.93	134
B	0.86	0.93	0.89	147
Г	0.96	0.95	0.96	128
Д	0.84	0.81	0.83	140
E	0.70	0.88	0.78	173
Ж	0.86	0.96	0.91	139
З	0.93	0.88	0.91	130
И	0.87	0.86	0.87	135
Й	0.90	0.92	0.91	140
K	0.91	0.93	0.92	138
Л	0.94	0.95	0.94	130
М	0.91	0.89	0.90	141
Н	0.90	0.86	0.88	153
O	0.95	0.93	0.94	140
П	0.82	0.90	0.86	143
P	0.94	0.89	0.92	148
C	0.98	0.93	0.95	154
T	0.95	0.83	0.89	137
У	0.93	0.97	0.95	166
Ф	0.86	0.94	0.90	139
X	0.91	0.91	0.91	146
Ц	0.89	0.88	0.88	135
Ч	0.98	0.84	0.91	140
Ш	0.83	0.92	0.88	130
Щ	0.92	0.81	0.86	134
Ъ	0.98	0.88	0.93	138
Ы	0.97	0.90	0.94	125
Ь	0.92	0.95	0.94	129
Э	0.94	0.94	0.94	138
Ю	0.96	0.92	0.94	139
Я	0.91	0.97	0.94	132
accuracy			0.90	4661
macro avg	0.90	0.90	0.90	4661
weighted avg	0.90	0.90	0.90	4661



Simple Sigmoid

```
In [36]: model_with_dropout_sigmoid = tf.keras.Sequential(
    [
        tf.keras.layers.Rescaling(1.0 / 255),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(num_classes, activation="sigmoid"),
    ]
)

model_with_dropout_sigmoid.compile(
    optimizer="adam",
```

```
loss=tf.keras.losses.CategoricalCrossentropy(from_logits=False),  
metrics=["accuracy"],  
)
```

```
In [37]: results_dropout_model_sigmoid = model_with_dropout_sigmoid.fit(  
        training_dataset, validation_data=validation_dataset, epochs=epochs, verbose=1  
)
```

```
Epoch 1/15  
271/271  11s 24ms/step - accuracy: 0.2328 - loss: 2.7474  
- val_accuracy: 0.6195 - val_loss: 1.4492  
Epoch 2/15  
271/271  2s 9ms/step - accuracy: 0.5747 - loss: 1.4423 -  
val_accuracy: 0.7998 - val_loss: 0.7406  
Epoch 3/15  
271/271  2s 9ms/step - accuracy: 0.6878 - loss: 1.0087 -  
val_accuracy: 0.8761 - val_loss: 0.4472  
Epoch 4/15  
271/271  2s 9ms/step - accuracy: 0.7468 - loss: 0.8223 -  
val_accuracy: 0.8877 - val_loss: 0.3758  
Epoch 5/15  
271/271  2s 9ms/step - accuracy: 0.7776 - loss: 0.7090 -  
val_accuracy: 0.9283 - val_loss: 0.2825  
Epoch 6/15  
271/271  2s 8ms/step - accuracy: 0.8071 - loss: 0.6127 -  
val_accuracy: 0.9339 - val_loss: 0.2432  
Epoch 7/15  
271/271  2s 9ms/step - accuracy: 0.8246 - loss: 0.5577 -  
val_accuracy: 0.9334 - val_loss: 0.2432  
Epoch 8/15  
271/271  2s 9ms/step - accuracy: 0.8329 - loss: 0.5203 -  
val_accuracy: 0.9413 - val_loss: 0.2272  
Epoch 9/15  
271/271  2s 9ms/step - accuracy: 0.8481 - loss: 0.4553 -  
val_accuracy: 0.9478 - val_loss: 0.1802  
Epoch 10/15  
271/271  2s 9ms/step - accuracy: 0.8572 - loss: 0.4344 -  
val_accuracy: 0.9491 - val_loss: 0.1806  
Epoch 11/15  
271/271  2s 9ms/step - accuracy: 0.8637 - loss: 0.4057 -  
val_accuracy: 0.9593 - val_loss: 0.1531  
Epoch 12/15  
271/271  2s 9ms/step - accuracy: 0.8649 - loss: 0.3887 -  
val_accuracy: 0.9626 - val_loss: 0.1374  
Epoch 13/15  
271/271  2s 9ms/step - accuracy: 0.8801 - loss: 0.3539 -  
val_accuracy: 0.9635 - val_loss: 0.1203  
Epoch 14/15  
271/271  2s 9ms/step - accuracy: 0.8862 - loss: 0.3202 -  
val_accuracy: 0.9630 - val_loss: 0.1122  
Epoch 15/15  
271/271  2s 9ms/step - accuracy: 0.8890 - loss: 0.3122 -  
val_accuracy: 0.9718 - val_loss: 0.1105
```

```
In [38]: # Compute predictions and true labels  
y_pred_probs = model_with_dropout_sigmoid.predict(test_dataset)
```

```

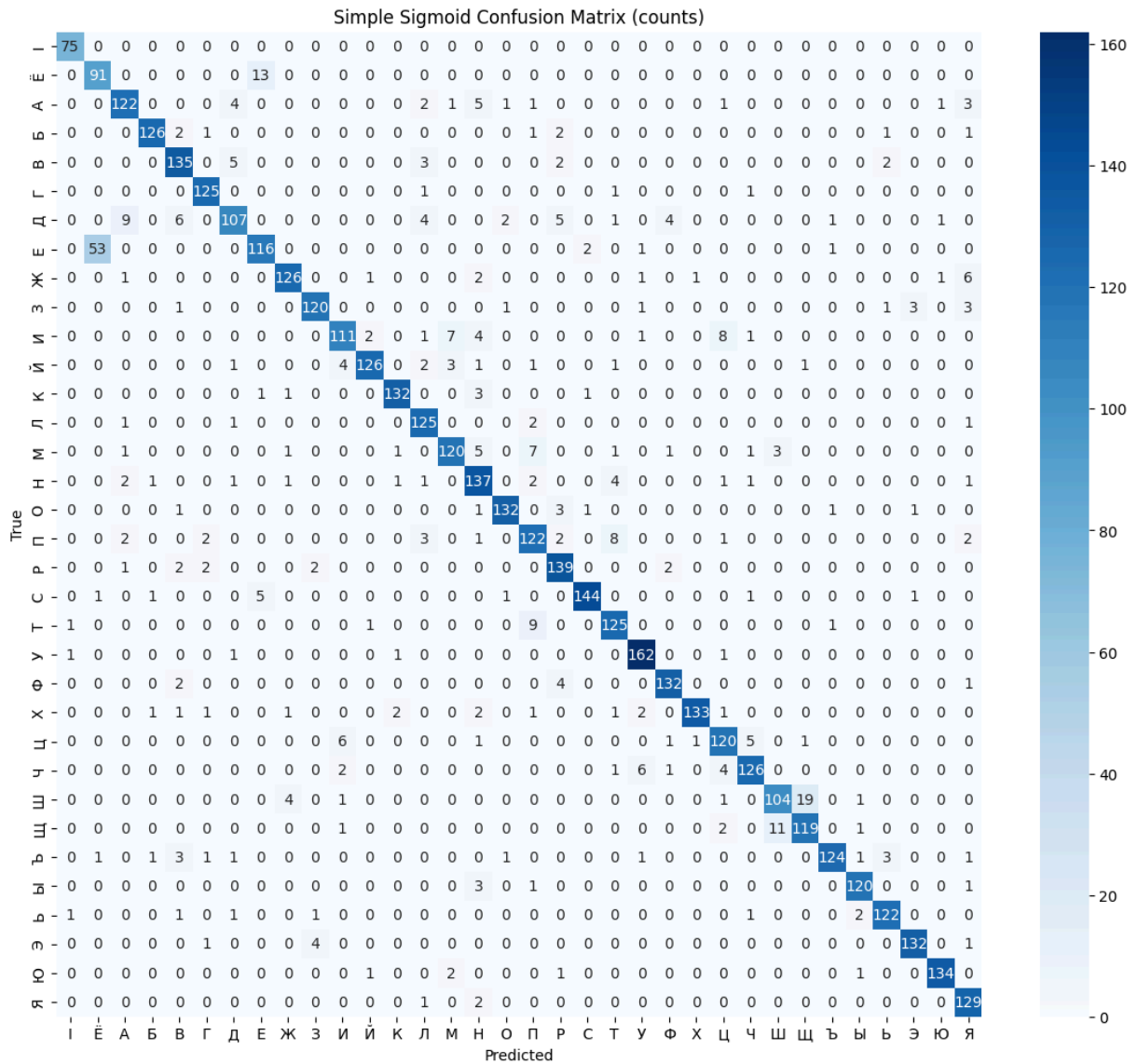
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.concatenate(
    [np.argmax(labels_batch, axis=1) for _, labels_batch in test_dataset], a
)

# Confusion matrix and classification report
cm = confusion_matrix(y_true, y_pred)
print(classification_report(y_true, y_pred, target_names=class_names, zero_c

# Plot confusion matrix (raw counts)
plt.figure(figsize=(14, 12))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names,
)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Simple Sigmoid Confusion Matrix (counts)")
plt.show()

```

	precision	recall	f1-score	support
I	0.96	1.00	0.98	75
Ё	0.62	0.88	0.73	104
A	0.88	0.87	0.87	141
Б	0.97	0.94	0.95	134
B	0.88	0.92	0.90	147
Г	0.94	0.98	0.96	128
Д	0.88	0.76	0.82	140
E	0.86	0.67	0.75	173
Ж	0.94	0.91	0.92	139
З	0.94	0.92	0.93	130
И	0.89	0.82	0.85	135
Й	0.96	0.90	0.93	140
K	0.96	0.96	0.96	138
Л	0.87	0.96	0.92	130
M	0.90	0.85	0.88	141
H	0.82	0.90	0.86	153
O	0.96	0.94	0.95	140
П	0.83	0.85	0.84	143
P	0.88	0.94	0.91	148
C	0.97	0.94	0.95	154
T	0.87	0.91	0.89	137
У	0.93	0.98	0.95	166
Ф	0.94	0.95	0.94	139
X	0.99	0.91	0.95	146
Ц	0.86	0.89	0.87	135
Ч	0.92	0.90	0.91	140
Ш	0.88	0.80	0.84	130
Щ	0.85	0.89	0.87	134
Ъ	0.97	0.90	0.93	138
Ы	0.95	0.96	0.96	125
Ь	0.95	0.95	0.95	129
Э	0.96	0.96	0.96	138
Ю	0.98	0.96	0.97	139
Я	0.86	0.98	0.91	132
accuracy			0.90	4661
macro avg	0.91	0.91	0.90	4661
weighted avg	0.91	0.90	0.90	4661



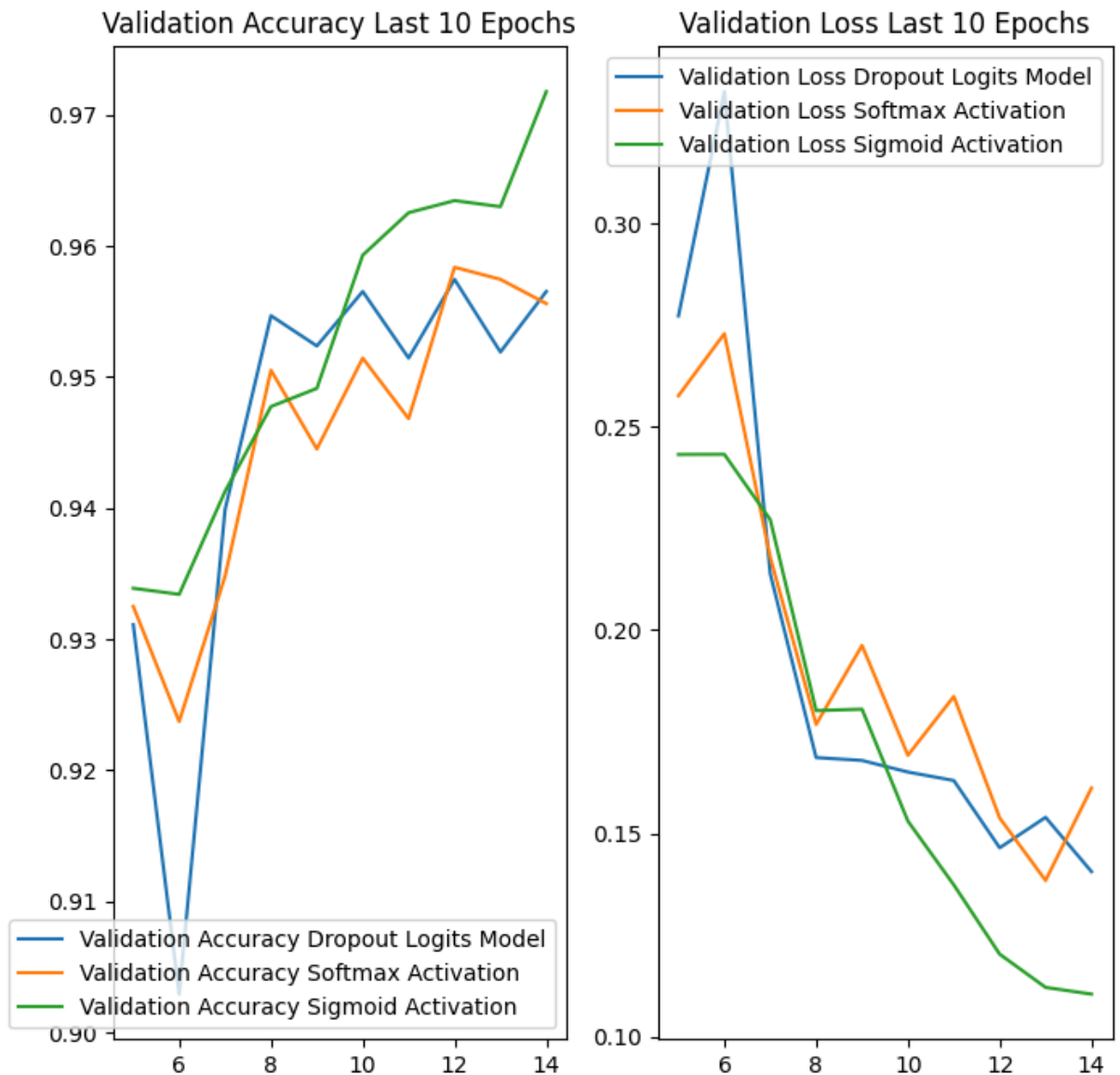
```
In [43]: plt.figure(figsize=(8, 8))

plt.subplot(1, 2, 1)
plt.plot(
    last_ten_epochs,
    results_dropout_model.history["val_accuracy"][-10:],
    label="Validation Accuracy Dropout Logits Model",
)
plt.plot(
    last_ten_epochs,
    results_dropout_model_softmax.history["val_accuracy"][-10:],
    label="Validation Accuracy Softmax Activation",
)
plt.plot(
    last_ten_epochs,
    results_dropout_model_sigmoid.history["val_accuracy"][-10:],
    label="Validation Accuracy Sigmoid Activation",
)
plt.legend(loc="lower right")
plt.title("Validation Accuracy Last 10 Epochs")
```

```

plt.subplot(1, 2, 2)
plt.plot(
    last_ten_epochs,
    results_dropout_model.history["val_loss"][-10:],
    label="Validation Loss Dropout Logits Model",
)
plt.plot(
    last_ten_epochs,
    results_dropout_model_softmax.history["val_loss"][-10:],
    label="Validation Loss Softmax Activation",
)
plt.plot(
    last_ten_epochs,
    results_dropout_model_sigmoid.history["val_loss"][-10:],
    label="Validation Loss Sigmoid Activation",
)
plt.legend(loc="upper right")
plt.title("Validation Loss Last 10 Epochs")
plt.show()

```



Consolidating Metrics Across All Models

```
In [40]: import pandas as pd

# Define all models with their names
models_dict = {
    "Main Model (Raw Logits)": model_main,
    "Dropout (32 Batch)": model_with_dropout,
    "Dropout (16 Batch)": model_with_dropout_16,
    "Dropout (64 Batch)": model_with_dropout_64,
    "Dropout (MSE Loss)": model_with_dropout_mse,
    "Dropout (Softmax)": model_with_dropout_softmax,
    "Dropout (Sigmoid)": model_with_dropout_sigmoid,
}

# Store metrics for each model
metrics_list = []

for model_name, model in models_dict.items():
    # Get predictions
    y_pred_probs = model.predict(test_dataset, verbose=0)
    y_pred = np.argmax(y_pred_probs, axis=1)
    y_true = np.concatenate(
        [np.argmax(labels_batch, axis=1) for _, labels_batch in test_dataset
        ])

    # Get classification report as dictionary
    report = classification_report(y_true, y_pred, target_names=class_names,

    # Extract macro metrics
    macro_precision = report['macro avg']['precision']
    macro_recall = report['macro avg']['recall']
    macro_f1 = report['macro avg']['f1-score']
    macro_accuracy = report['accuracy'] # Overall accuracy

    metrics_list.append({
        'Model': model_name,
        'Macro Accuracy': macro_accuracy,
        'Macro Precision': macro_precision,
        'Macro Recall': macro_recall,
        'Macro F1': macro_f1,
    })

# Create dataframe
metrics_df = pd.DataFrame(metrics_list)
print(metrics_df.round(2).to_string(index=False))
metrics_df.round(2).to_csv("metrics.csv", index=False)
```

	Model	Macro Accuracy	Macro Precision	Macro Recall	Macro F1
0.87	Main Model (Raw Logits)	0.87	0.88	0.87	
0.89	Dropout (32 Batch)	0.89	0.90	0.89	
0.89	Dropout (16 Batch)	0.89	0.90	0.89	
0.90	Dropout (64 Batch)	0.90	0.90	0.90	
0.91	Dropout (MSE Loss)	0.91	0.91	0.91	
0.90	Dropout (Softmax)	0.90	0.90	0.90	
0.90	Dropout (Sigmoid)	0.90	0.91	0.91	

Κώδικας Αξιολόγησης Προεκπαιδευμένου Μοντέλου CoMNIST

```
In [21]: import os
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'
import time
import tensorflow as tf
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import os
import string
import numpy as np
import tensorflow as tf
from keras.layers import (
    Dense,
    Convolution2D,
    MaxPooling2D,
    Flatten,
)
from keras.models import Sequential
from PIL import Image, ImageOps
import matplotlib.pyplot as plt
```

```
In [8]: RUSSIAN_WEIGHTS = "api/weights/comnist_keras_ru.hdf5"
TEST_DIR = "images/Cyrillic_test"
SIZE = 32
nb_classes = 34
batch_size = 32
img_height = 32
img_width = 32
```

```
In [9]: # Load the datasets

# test_dataset = tf.keras.utils.image_dataset_from_directory(
#     TEST_DIR,
#     seed=123,
#     image_size=(img_height, img_width),
#     batch_size=batch_size,
#     labels="inferred",
# )

class_names = sorted([d for d in os.listdir(TEST_DIR) if os.path.isdir(os.path.join(TEST_DIR, d))])
name_to_index = {name: i for i, name in enumerate(class_names)}
print(class_names)
print(name_to_index)
```

```
[ 'I', 'Ё', 'А', 'Б', 'В', 'Г', 'Д', 'Е', 'Ж', 'З', 'И', 'Й', 'К', 'Л', 'М',
  'Н', 'О', 'П', 'Р', 'С', 'Т', 'У', 'Ф', 'Х', 'Ц', 'Ч', 'Ш', 'Щ', 'Ъ', 'Ы',
  'Ь', 'Э', 'Ю', 'Я']
{ 'I': 0, 'Ё': 1, 'А': 2, 'Б': 3, 'В': 4, 'Г': 5, 'Д': 6, 'Е': 7, 'Ж': 8,
  'З': 9, 'И': 10, 'Й': 11, 'К': 12, 'Л': 13, 'М': 14, 'Н': 15, 'О': 16, 'П':
  17, 'Р': 18, 'С': 19, 'Т': 20, 'У': 21, 'Ф': 22, 'Х': 23, 'Ц': 24, 'Ч': 25,
  'Ш': 26, 'Щ': 27, 'Ъ': 28, 'Ы': 29, 'Ь': 30, 'Э': 31, 'Ю': 32, 'Я': 33}
```

```
In [10]: from api.model import load_letter_predictor
         from IPython.display import display

         commnist_predictor = load_letter_predictor(weight=RUSSIAN_WEIGHTS, nb_classes=34)
         img_copy = Image.open("images/Cyrillic_test/A/5a2f3c19c27bb.png")
         blank = Image.new("L", img_copy.size, color=255)
         img_copy.paste(blank, (0, 0), mask=img_copy)
         img_copy = img_copy.convert("L")
         img_copy = ImageOps.invert(img_copy)
         display(img_copy)
```

Loaded weights from file api/weights/commnist_keras_ru.hdf5
Created model and loaded weights from file



```
In [11]: from api.image_proc import crop_letters

         for i, letter in enumerate(crop_letters(img_copy)):
             predicted_letter = commnist_predictor(letter, 1)
             print(f"Predicted letter: {predicted_letter}")
```

Predicted letter: ['А']

```
In [12]: y_true = []
         y_pred = []

         for cls in class_names:
             folder = os.path.join(TEST_DIR, cls)
             for fn in os.listdir(folder):
                 img_path = os.path.join(folder, fn)
                 img = Image.open(img_path)
```

```

        # Add a white background to the image

blank = Image.new("L", img.size, color=255)
img.paste(blank, (0, 0), mask=img)
img = img.convert("L")
img = ImageOps.invert(img)

# Get negative of image in case it is white on black
# img_np = np.array(img)
# if np.mean(img_np) < 128:

pred_list = commnist_predictor(img, nb_output=1) # returns list li
pred_char = pred_list[-1]
# convert true/pred to integer class indices
y_true.append(name_to_index[cls])
# find predicted class index in class_names (ensure char == folder r
# if your folder names are letter characters use:
y_pred.append(name_to_index.get(pred_char, -1))

```

```
In [16]: print("Accuracy:", accuracy_score(y_true, y_pred)*100,"%")
```

Accuracy: 90.15232782664664 %

```
In [17]: print(classification_report(y_true, y_pred, target_names=class_names))
```

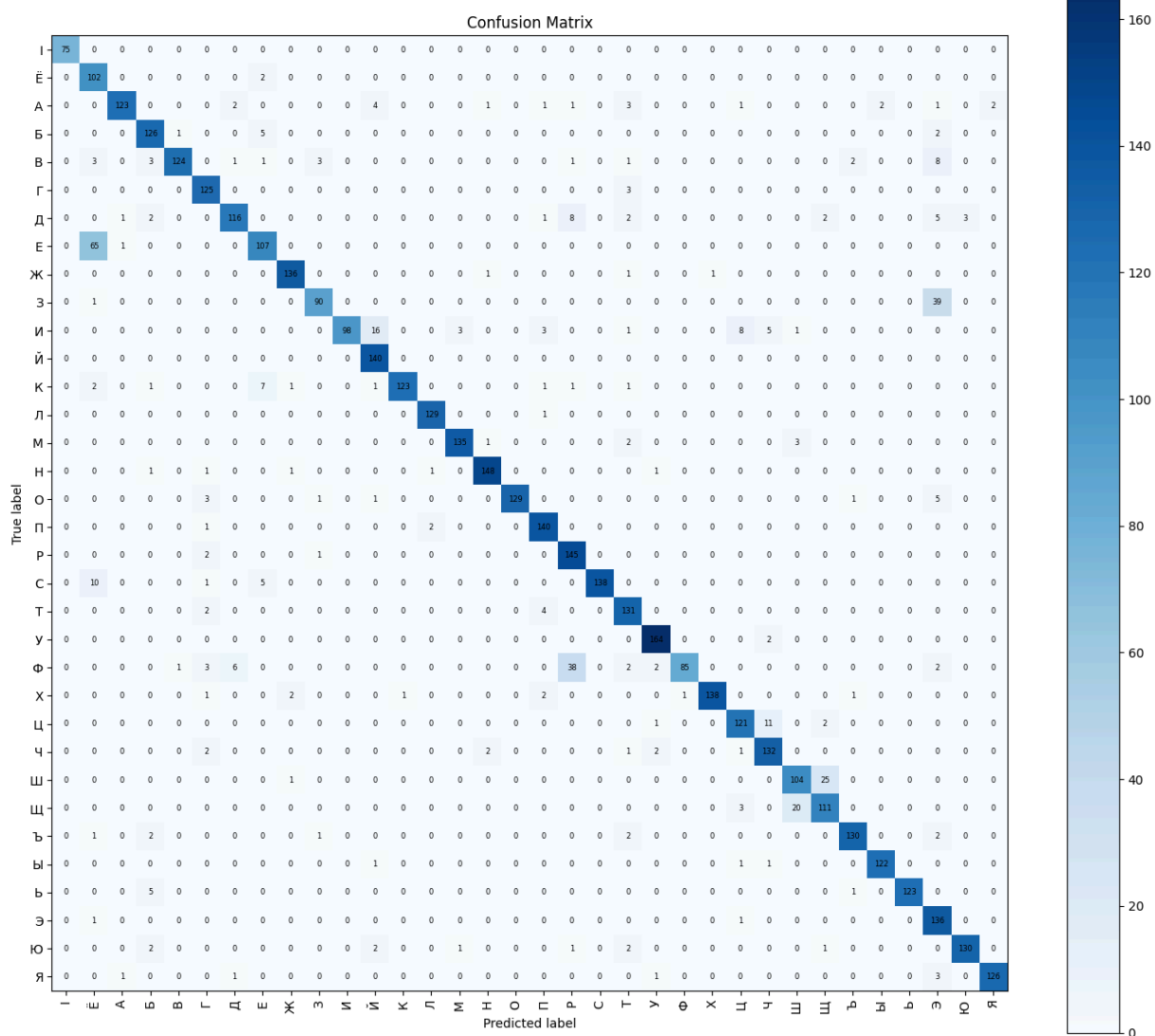
	precision	recall	f1-score	support
I	1.00	1.00	1.00	75
Ě	0.55	0.98	0.71	104
A	0.98	0.87	0.92	141
Б	0.89	0.94	0.91	134
B	0.98	0.84	0.91	147
Г	0.89	0.98	0.93	128
Д	0.92	0.83	0.87	140
E	0.84	0.62	0.71	173
Ж	0.96	0.98	0.97	139
З	0.94	0.69	0.80	130
И	1.00	0.73	0.84	135
Й	0.85	1.00	0.92	140
K	0.99	0.89	0.94	138
Л	0.98	0.99	0.98	130
M	0.97	0.96	0.96	141
H	0.97	0.97	0.97	153
O	1.00	0.92	0.96	140
П	0.92	0.98	0.95	143
P	0.74	0.98	0.85	148
C	1.00	0.90	0.95	154
T	0.86	0.96	0.91	137
У	0.96	0.99	0.97	166
Ф	0.99	0.61	0.76	139
X	0.99	0.95	0.97	146
Ц	0.89	0.90	0.89	135
Ч	0.87	0.94	0.91	140
Ш	0.81	0.80	0.81	130
Щ	0.79	0.83	0.81	134
Ъ	0.96	0.94	0.95	138
Ы	0.98	0.98	0.98	125
Ь	1.00	0.95	0.98	129
Э	0.67	0.99	0.80	138
Ю	0.98	0.94	0.96	139
Я	0.98	0.95	0.97	132
accuracy			0.90	4661
macro avg	0.91	0.90	0.90	4661
weighted avg	0.92	0.90	0.90	4661

```
In [22]: cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(14, 12))
plt.imshow(cm, interpolation='nearest', cmap='Blues')
plt.colorbar()
plt.xticks(np.arange(len(class_names)), class_names, rotation=90)
plt.yticks(np.arange(len(class_names)), class_names)
plt.xlabel('Predicted label')
plt.ylabel('True label')
plt.title('Confusion Matrix')
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
```



```
plt.text(j, i, cm[i, j], ha='center', va='center', color='black', fontweight='bold')
plt.tight_layout()
plt.show()
```



In []: